A Major Project report

On

BUILDING SEARCH ENGINE USING MACHINE LEARNING

A dissertation Submitted in partial fulfillment of the requirement

for the

Award of the degree of

BACHELOR OF TECHNOLOGY

In

ELECTRONICS AND COMMUNICATION ENGINEERING

By

J. Anil Kumar	**21N31A0491**
K. Abhinay Reddy	**21N31A0499**
G. Srinivasa Reddy	**21N31A0473**

Under the esteemed guidance of

Dr.R.Chinna Rao

Associate Professor

DEPARTMENT OF ELECTRONICS & COMMUICATION ENGINEERING

MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY

Autonomous Institution – UGC, Govt. of India

(Affiliated to JNTU, Hyderabad, Approved by AICTE - Accredited by NBA & NAAC – 'A' Grade)

Maisammaguda, Dhulapally (Post Via Hakimpet), Secunderabad – 500

2021-2025

2021-2025

CERTIFICATE

This is to certify that the **MAJOR PROJECT** work entitled **"BUILDING SEARCH ENGINE USING MACHINE LEARNING"** is carried out by **J. Anil Kumar (21N31A0491), K. Abhinay Reddy (21N31A0499)** and **G. Srinivasa Reddy (21N31A0473)**, in partial fulfillment for the award of degree of ***BACHELOR OF TECHNOLOGY*** in Electronics and Communication Engineering, Jawaharlal Nehru Technological University, Hyderabad during the academic year 2024-2025.

INTERNAL GUIDE	HEAD OF THE DEPARTMENT
Dr.R.Chinna Rao Associate Professor	Dr.K. MALLIKARJUNA LINGAM PROFESSOR & HOD

EXTERNAL

DECLARATION

We hereby declare that the Major Project titled **"Building Search Engine Using Machine Learning"** submitted to Malla Reddy College of Engineering and Technology (UGC Autonomous), affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) forthe award of the degree of Bachelor of Technology in Electronics and Communication Engineering is a result of original research carried-out in this thesis. It is further declared that the Major Project report or any part there of has not been previously submitted to any University or Institute for the award of degree or diploma.

J. Anil Kumar - 21N31A0491

K. Abhinay Reddy - 21N31A0499

G. Srinivasa Reddy - 21N31A0473

ACKNOWLEDGEMENTS

First and foremost, we would like to thank God for giving strength to complete this work. We feel ourselves honoured and privileged to place our warm salutation to our college Malla Reddy College of Engineering and Technology (UGC-Autonomous) and our Director Dr. VSK Reddy, Principal Dr. S. Srinivasa Rao who gave us the opportunity to have experience in engineering and profound technical knowledge.

We express our heartiest thanks to Prof P. Sanjeeva Reddy, Director of the Department of Electronics and Communication Engineering and Dr.K. Mallikarjuna Lingam, Head of the Department of Electronics and communication engineering for encouraging us in every aspect of our project and helping us realize our full potential.

We would like to thank our Project Coordinator Mrs.Neha Thakur, Associate Professor, Department of Electronics and Communication Engineering for her valuable guidance and encouragement during my project work.

We would like to thank our internal guide Dr.R.Chinna Rao, Associate Professor, Department of Electronics and Communication Engineering for her regular guidance and constant encouragement. We are extremely grateful to her valuable suggestions and unflinching cooperation throughout project work.

We would like to thank Dr. T. Venugopal, Dean of Academics, who in spite of being busy with his duties took time to guide and keep us on the correct path.

We would also like to thank all the supporting staff of the Department of Electronics and Communication Engineering and all other departments who have been helpful directly orindirectly in making our project a success.

We are extremely grateful to our parents for their blessings and prayers for the completion of our project that gave us strength to do our project.

By

J.Anil Kumar (21N31A0491)
K.Abhinay Reddy (21N31A0499)
G.Srinivasa Reddy (21N31A0473)

CONTENTS

LIST OF FIGURES

ABSTRACT

The web is the huge and most extravagant wellspring of data. To recover the information from the World Wide Web, Search Engines are commonly utilized. Search engines provide a simple interface for searching for user query and displaying results in the form of the web address of the relevant web page, but using traditional search engines has become very challenging to obtain suitable information. This project proposed a Machine Learning techniques for search engine development that will give more relevant web pages at top for user queries.

KEYWORDS-Search Engine, Machine Learning.

1 .INTRODUCTION

World Wide Web is actually a web of individual systems and servers which are connected with different technology and methods. Every site comprises the heaps of site pages that are being made and sent on the server. So if a user needs something, then he or she needs to type a keyword. Keyword is a set of words extracted from user search input. Search input given by a user may be syntactically incorrect. Here comes the actual need for search engines. Search engines provide you a simple interface to search user queries and display the results.

1) Web crawler Web crawlers help in collecting data about a website and the links related to them. We are only using web crawlers for collecting data and information from WWW and storing it in our database.

2) Indexer Indexer which arranges each term on each web page and stores the subsequent list of terms in a tremendous repository.

3) Query Engine It is mainly used to reply to the user's keyword and show the effective outcome for their keyword. In the query engine, the Page ranking algorithm ranks the URL by using different algorithms in the query engine.

4)This paper utilizes Machine Learning Techniques to discover the utmost suitable web address for the given keyword. The output of the PageRank algorithm is given as input to the machine learning algorithm.

5)The section II discusses the related work in search engine and PageRank algorithm. In section III Objective is explained. Section IV deals with a proposed system which is based on machine learning technique and section V contains the conclusion.

2. LITERATURE SURVEY

1) Weighted page rank algorithm based on in-out weight of webpages

AUTHORS: Kalyani Desikan, B. Jaganathan.

In its classical formulation, the well known page rank algorithm ranks web pages only based on in-links between web pages. We propose a new in-out weight based page rank algorithm. In this paper, we have introduced a new weight matrix based on both the in-links and out-links between web pages to compute the page ranks. We have illustrated the working of our algorithm using a web graph. We notice that the page rank values of the web pages computed using the original page rank algorithm and our proposed algorithm are comparable. Moreover, our algorithm is found to be efficient with respect to the time taken to compute the page rank values.

2) Web page Ranking Using Machine Learning Approach.

AUTHORS:Junaid Khan, Arunima Jaiswal.

One of the key components which ensures the acceptance of web search service is the web page ranker – a component which is said to have been the main contributing factor to the early successes of Google. It is well established that a machine learning method such as the Graph Neural Network(GNN) is able to learn and estimate Google's page ranking algorithm. This project shows that the GNN can successfully learn many other web page ranking methods e.g. TrustRank, HITS and OPIC. Experimental results show that GNN may be suitable to learn any arbitrary web page ranking scheme. The significance of this observation lies in the fact that it is possible to learn ranking schemes for which no algorithmic solution exists or is known.

3) Review of features and machine learning techniques for web searching.

AUTHORS:Neha Sharm ,Narendra Kohli

As the amount of information is growing rapidly on world wide web, it has become very difficult to get relevant information using traditional search engines within a stipulated time.The main reasons for irrelevant search results are the lack of understanding of user's search intention or user's preferences, keyword based searching, short queries. In this paper, we will study different features that are used in information retrieval. We will also discuss various

machine learning techniques that are helpful in deciding the relevance of web page to user. We have done classification on the basis of features. In the end we will compare different techniques and their pros and cons are also discussed.

3. SYSTEM ANALYSIS

3.1 Feasibility Study

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are,

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

3.1.1 Economical Feasibility

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

3.1.2 Technical Feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

3.1.3 Social Feasibility

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make the system.

3.2 Existing Systems

Shodan is that the pc programme for everything on the online. whereas Google and different search engines index exclusively the net, Shodan indexes almost everything else — internet cams, water treatment facilities, yachts, medical devices, traffic lights, wind turbines, registration code readers, smart TVs, refrigerators, one thing and everything you will in all probability imagine that's clogged into the online .

Services running on open ports announce themselves, of course, with banners. A banner publically declares to the whole internet what service it offers and therefore the thanks to act with it. Alternative services on different ports give service-specific information that is not a guarantee that the written banner is true or real. In most cases, it is, and in any event mercantilism a deliberately dishonest banner is security by obscurity. Some enterprises block Shodan from locomotion their network, and Shodan honors such requests. However, attackers don't need Shodan to hunt out vulnerable devices connected to your network. obstruction Shodan might stop from short embarrassment, but it's unlikely to boost your security posture.

3.3 Disadvantages

- Sometimes it takes too much time to display relevant, valuable and informative content.
- Search engines especially Google, frequently update their algorithm,it is very difficult to find the algorithm in which Google runs.

3.4 Proposed Systems

The programme is trained victimization two supervised machine learning algorithms particularly choice based mostly and review based. The tags/weights are calculated to rank the links within the coaching data-set. Each the algorithms follow the inclusion of various heuristics for identical. The load of the link is decided by the frequency of the keyword in content of the link and also the position wherever it happens. Also, heuristics like whether or not the keyword is written in daring or italics; position wherever it happens, for e.g. in page title, headings, data etc; and also the variety of outgoing links having keyword within the address.

3.5 Advantages

The programmer is trained victimization two supervised machine learning algorithms particularly choice based mostly and review based. The tags/weights are calculated to rank the links within the coaching data-set. Each the algorithms follow the inclusion of various heuristics for identical.

4. SYSTEM REQUIREMENTS

4.1 Software Requirement Specification

4.1.1 Hardware Requirements

- **System** : Pentium IV 2.4 GHz.
- **Hard Disk** : 40 GB.
- **Floppy Drive** : 1.44 MB.
- **Monitor** : 14’ Colour Monitor.
- **Mouse** : Optical Mouse.
- **RAM** : 512 MB.

4.1.2 Software Requirements

- **Operating system** : Windows 7 Ultimate.
- **Coding Language** : Python.
- **Front-End** : Python.
- **Designing** : HTML, CSS, JavaScript.
- **Data Base** : MySQL.

5. SYSTEM DESIGN

5.1 System Architecture

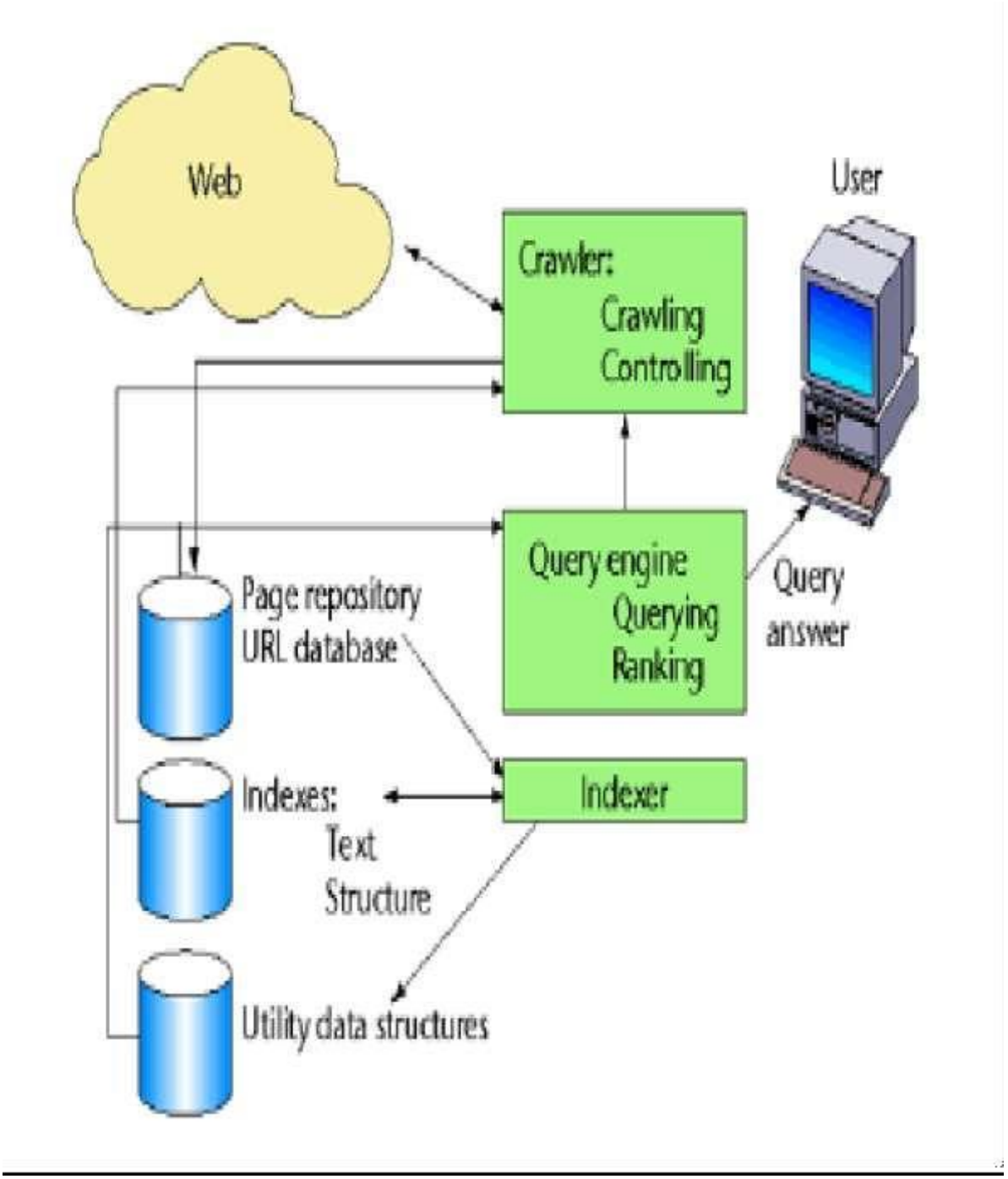


Fig No. 1.1

Data Flow Diagram

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.

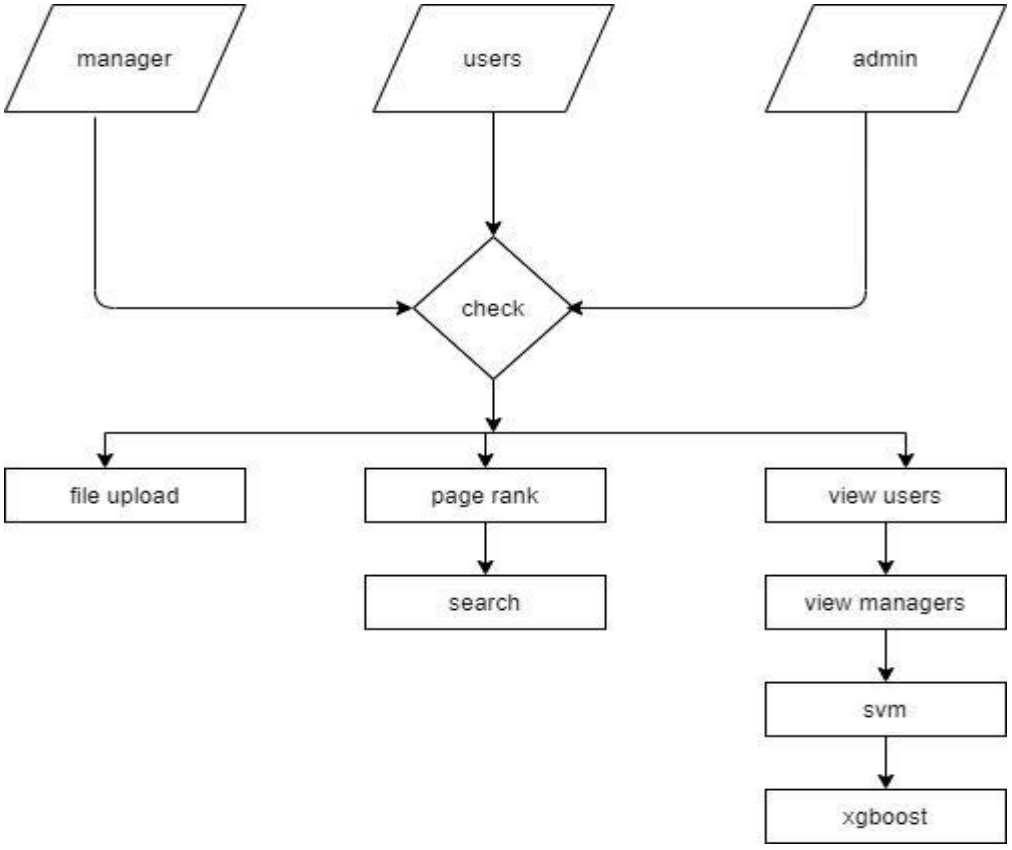


Fig No. 1.2

5.2 UML Diagrams

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

5.2.1 Goals

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

5.2.2 Use Case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

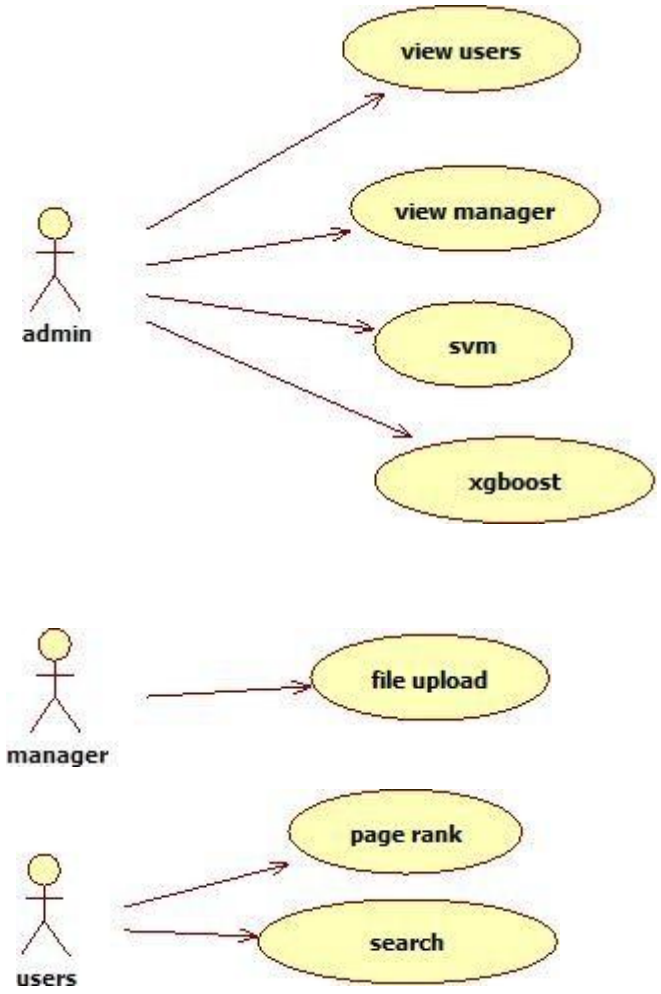


Fig No. 1.3

5.2.3 Class Diagram

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

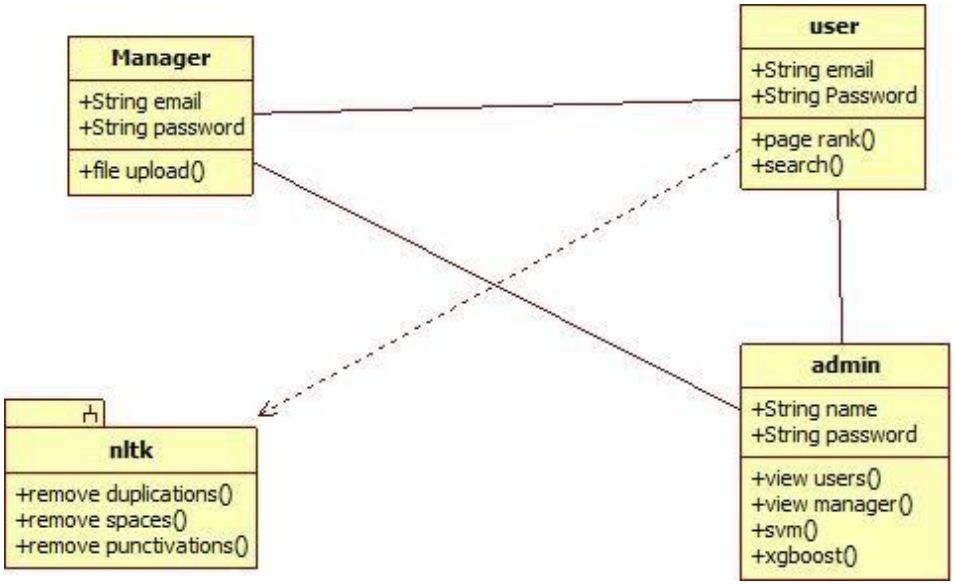


Fig No. 1.4

5.2.4 Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

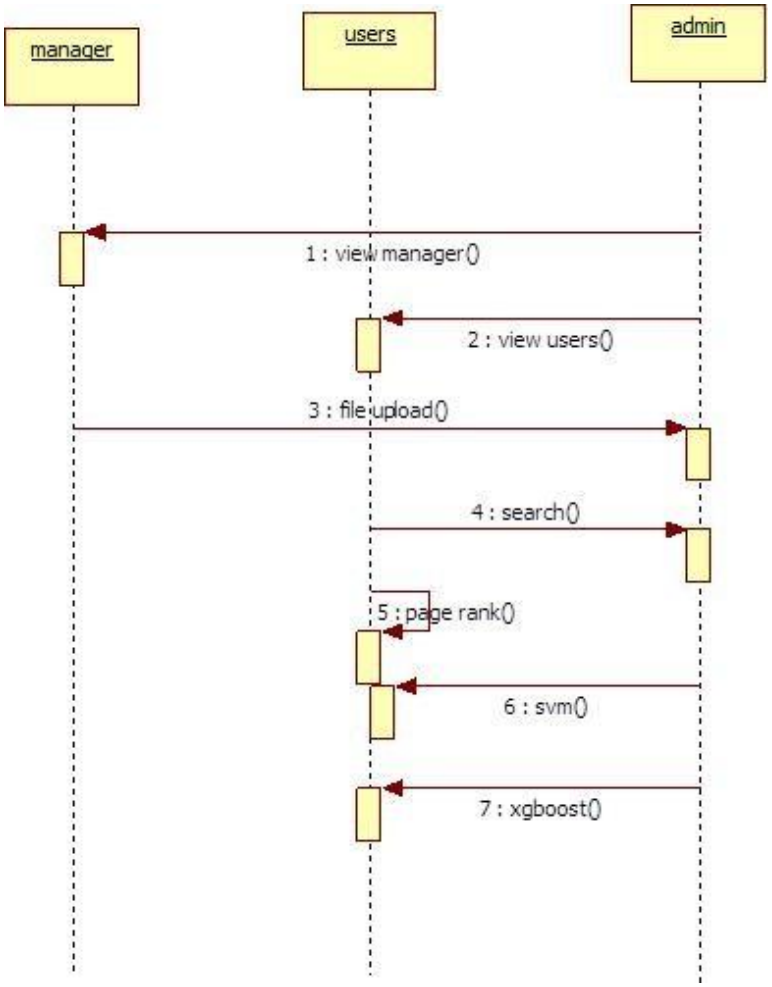


Fig No. 1.5

5.2.5 Activity Diagram

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control

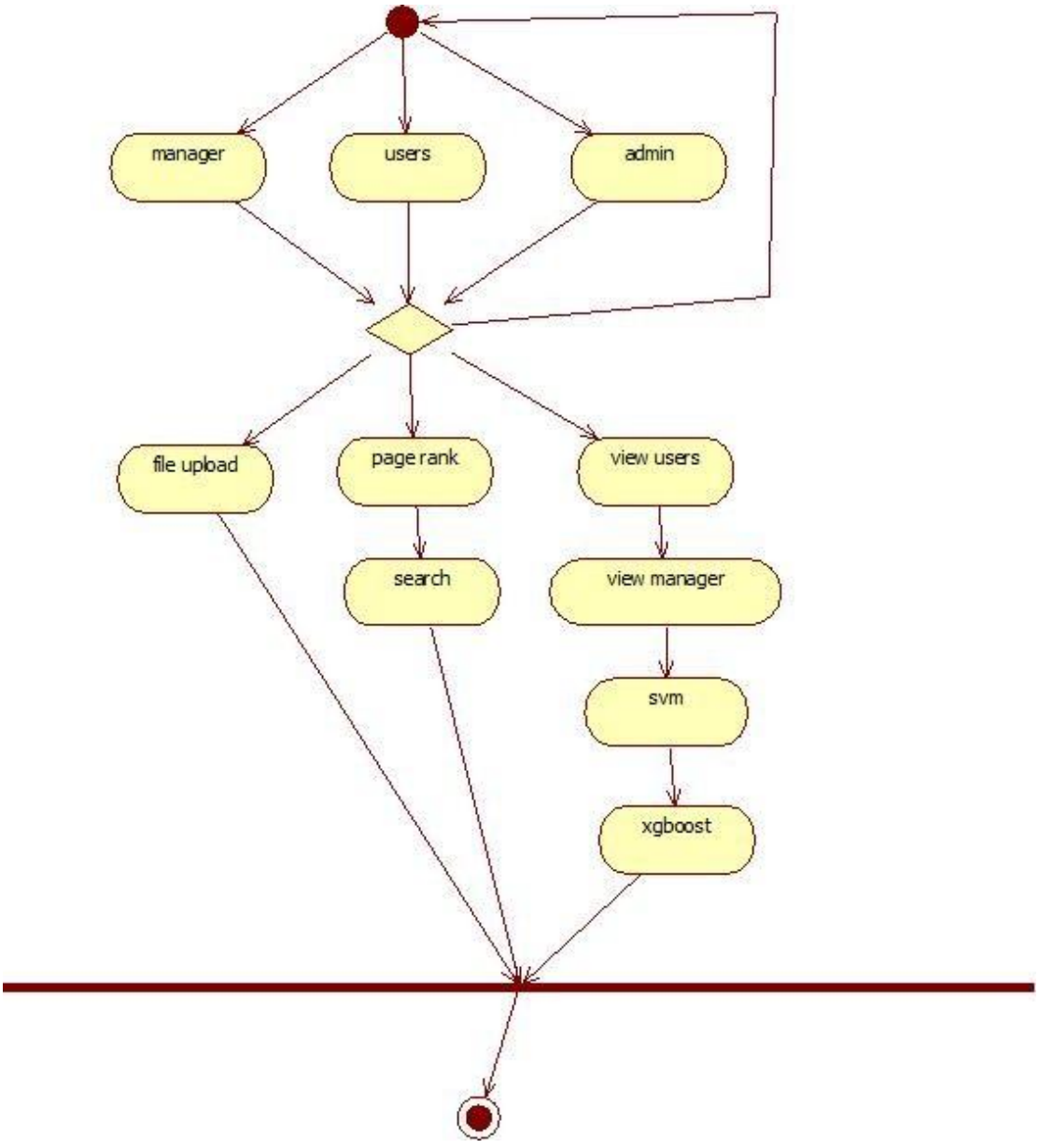


Fig No. 1.6

6. SOFTWARE ENVIRONMENT

6.1 Introduction to python

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An interpreted language, Python has a design philosophy that emphasizes code readability (notably using whitespace indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. CPython, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. CPython is managed by the non-profit Python Software Foundation. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object- oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

Interactive Mode Programming

Invoking the interpreter without passing a script file as a parameter brings up the following prompt −

```
$ python
Python 2.4.3 (#1, Nov 11 2010, 13:34:43)
[GCC 4.1.2 20080704 (Red Hat 4.1.2-48)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Type the following text at the Python prompt and press the Enter −

```
>>> print "Hello, Python!"
```

If you are running new version of Python, then you would need to use print statement with parenthesis as in print ("Hello, Python!");. However in Python version 2.4.3, this produces the following result −

```
Hello, Python!
```

Script Mode Programming

Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.

Let us write a simple Python program in a script. Python files have extension .py. Type the following source code in a test.py file −

Live Demo

```
print "Hello, Python!"
```

We assume that you have Python interpreter set in PATH variable. Now, try to run this program as follows −$ python test.py

This produces the following result −

```
Hello, Python!
```

Let us try another way to execute a Python script. Here is the modified test.py file −

Live Demo

```
#!/usr/bin/python
print "Hello, Python!"
```

We assume that you have Python interpreter available in /usr/bin directory. Now, try to run this program as follows −

```
$ chmod +x test.py     # This is to make file executable
$./test.py
```

This produces the following result −

```
Hello, Python!
```

Python Identifiers

A Python identifier is a name used to identify a variable, function, class, module or other object. An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).

Python does not allow punctuation characters such as @, $, and % within identifiers. Python is a case sensitive programming language. Thus, Manpower and manpower are two different identifiers in Python.

Here are naming conventions for Python identifiers −

Class names start with an uppercase letter. All other identifiers start with a lowercase letter. Starting an identifier with a single leading underscore indicates that the identifier is private. Starting an identifier with two leading underscores indicates a strongly private identifier.

If the identifier also ends with two trailing underscores, the identifier is a language-defined special name.

Reserved Words

The following list shows the Python keywords. These are reserved words and you cannot use them as constant or variable or any other identifier names. All the Python keywords contain lowercase letters only.

and	exec	not
assert	finally	or
break	for	pass
class	from	print
continue	global	raise
def	if	return
del	import	try
elif	in	while
else	is	with
except	lambda	yield

Lines and Indentation

Python provides no braces to indicate blocks of code for class and function definitions or flow control. Blocks of code are denoted by line indentation, which is rigidly enforced.

The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount. For example −

```
if True:
   print "True"
else:
   print "False"
```

However, the following block generates an error

```
– if True:
print   "Answer"
print       "True"
else:
print   "Answer"
print "False"
```

Thus, in Python all the continuous lines indented with same number of spaces would form a block.

The following example has various statement blocks −

Note − Do not try to understand the logic at this point of time. Just make sure you understood various blocks even if they are without braces.

```
#!/usr/bin/python
import sys
try:
   # open file stream
   file = open(file_name, "w")
except IOError:
   print "There was an error writing to", file_name
   sys.exit()
print "Enter '", file_finish,
print "' When finished"
while file_text != file_finish:
   file_text = raw_input("Enter text: ")
```

```
  if file_text == file_finish:
  # close the file file.close  break
  file.write(file_text)
  file.write("\n")
file.close()
file_name = raw_input("Enter filename: ")
if len(file_name) == 0:
  print "Next time please enter
        something" sys.exit()
try:
  file = open(file_name, "r")
except IOError:
  print "There was an error reading file"
  sys.exit()
file_text = file.read() file.close()
print file_text
```

Multi-Line Statements

Statements in Python typically end with a new line. Python does, however, allow the use of the line continuation character (\) to denote that the line should continue. For example −

```
total = item_one + \
      item_two + \
      item_three
```

Statements contained within the [], {}, or () brackets do not need to use the line continuation character. For example −

```
days = ['Monday', 'Tuesday', 'Wednesday',
      'Thursday', 'Friday']
```

Quotation in Python

Python accepts single ('), double (") and triple (''' or """) quotes to denote string literals, as long as the same type of quote starts and ends the string.

The triple quotes are used to span the string across multiple lines. For example, all the following are legal −

```
word = 'word'
```

```
sentence = "This is a
        sentence." paragraph =
"""This is a paragraph. It is
made up of multiple lines and sentences."""
```

Comments in Python

A hash sign (#) that is not inside a string literal begins a comment. All characters after the # and up to the end of the physical line are part of the comment and the Python interpreter ignores them.

Live Demo

```
#!/usr/bin/pytho
n # First
comment
print "Hello, Python!" # second comment
```

This produces the following result −

```
Hello, Python!
```

You can type a comment on the same line after a statement or expression −

```
name = "Madisetti" # This is again comment
```

You can comment multiple lines as follows −

```
# This is a comment.
# This is a comment, too.
# This is a comment, too.
# I said that already.
```

Following triple-quoted string is also ignored by Python interpreter and can be used as a multiline comments:

```
'''
This is a multiline
comment.
'''Using Blank Lines
```

A line containing only whitespace, possibly with a comment, is known as a blank line and Python totally ignores it.

In an interactive interpreter session, you must enter an empty physical line to terminate a multiline statement.

Waiting for the User

The following line of the program displays the prompt, the statement saying “Press the enter key to exit”, and waits for the user to take action −

```
#!/usr/bin/python
raw_input("\n\nPress the enter key to exit.")
```

Here, "\n\n" is used to create two new lines before displaying the actual line. Once the user presses the key, the program ends. This is a nice trick to keep a console window open until the user is done with an application.

Multiple Statements on a Single Line

The semicolon (;) allows multiple statements on the single line given that neither statement starts a new code block. Here is a sample snip using the semicolon.

```
import sys; x = 'foo'; sys.stdout.write(x + '\n')
```

Multiple Statement Groups as Suites

A group of individual statements, which make a single code block are called suites in Python. Compound or complex statements, such as if, while, def, and class require a header line and a suite.

Header lines begin the statement (with the keyword) and terminate with a colon (:) and are followed by one or more lines which make up the suite. For example −

```
if expression :
   suite
elif expression :
   suit
e else :
   suite
```

Command Line Arguments

Many programs can be run to provide you with some basic information about how they should be run. Python enables you to do this with -h −

```
$ python -h
usage: python [option] ... [-c cmd | -m mod | file | -] [arg] ...
Options and arguments (and corresponding environment variables):
-c cmd : program passed in as string (terminates option list)
-d     : debug output from parser (also PYTHONDEBUG=x)
```

-E : ignore environment variables (such as PYTHONPATH)

-h : print this help message and exit

You can also program your script in such a way that it should accept various options. Command Line Arguments is an advanced topic and should be studied a bit later once you have gone through rest of the Python concepts.

Python Lists

The list is a most versatile datatype available in Python which can be written as a list of comma- separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example −

```
list1 = ['physics', 'chemistry', 1997, 2000];
list2 = [1, 2, 3, 4, 5 ];
list3 = ["a", "b", "c", "d"]
```

Similar to string indices, list indices start at 0, and lists can be sliced, concatenated and so on.

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5 );
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing

− tup1 = ();

To write a tuple containing a single value you have to include a comma, even though there is only one value −tup1 = (50,);

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

Accessing Values in Tuples

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

Live Demo

```
#!/usr/bin/python

tup1 = ('physics', 'chemistry', 1997, 2000);

tup2 = (1, 2, 3, 4, 5, 6, 7 );

print "tup1[0]: ", tup1[0];

print "tup2[1:5]: ", tup2[1:5];
```

When the above code is executed, it produces the following result −

```
tup1[0]:  physics

tup2[1:5]:  [2, 3, 4, 5]
```

Updating Tuples

Accessing Values in Dictionary

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example −

Live Demo

```
#!/usr/bin/python

dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

print "dict['Name']: ", dict['Name']

print "dict['Age']: ", dict['Age']
```

When the above code is executed, it produces the following result −

```
dict['Name']: Zara

dict['Age']:  7
```

If we attempt to access a data item with a key, which is not part of the dictionary, we get an error as follows − Live Demo

```
#!/usr/bin/python

dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

print "dict['Alice']: ", dict['Alice']
```

When the above code is executed, it produces the following result −

```
dict['Alice']:

Traceback (most recent call last):

 File "test.py", line 4, in <module>

print  "dict['Alice']:  ",  dict['Alice'];

KeyError: 'Alice'
```

Updating Dictionary

You can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example −

Live Demo

```
#!/usr/bin/python
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry
print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

When the above code is executed, it produces the following result −

```
dict['Age']: 8
dict['School']: DPS School
```

Delete Dictionary Elements

You can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the del statement. Following is a simple example −

Live Demo

```
#!/usr/bin/python
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'} del
dict['Name']; # remove entry with key 'Name'
dict.clear(); # remove all entries in dict
del dict ; # delete entire
dictionary print "dict['Age']: ",
dict['Age']
print "dict['School']: ", dict['School']
```

This produces the following result. Note that an exception is raised because after del dict dictionary does not exist any more −

```
dict['Age']:
Traceback (most recent call last):
File "test.py", line 8, in <module>
print "dict['Age']: ", dict['Age'];
```

TypeError: 'type' object is unsubscriptable

Note − del() method is discussed in subsequent section.

Properties of Dictionary Keys: Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects. However, same is not true for the keys.

There are two important points to remember about dictionary keys −

(a) More than one entry per key not allowed. Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins. For example –

(b) Live Demo

```
#!/usr/bin/python

dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'}
print "dict['Name']: ", dict['Name']
```

When the above code is executed, it produces the following result −

```
dict['Name']:  Manni
```

(b) Keys must be immutable. Which means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed. Following is a simple example −

Live Demo

```
#!/usr/bin/python
dict = {['Name']: 'Zara', 'Age': 7}
print "dict['Name']: ", dict['Name']
```

When the above code is executed, it produces the following result −

```
Traceback (most recent call last):
  File "test.py", line 3, in <module>
    dict = {['Name']: 'Zara', 'Age': 7};
TypeError: unhashable type: 'list'
```

Tuples are immutable which means you cannot update or change the values of tuple elements. You are able to take portions of existing tuples to create new tuples as the following example demonstrates −

Live Demo

```
#!/usr/bin/python
tup1 = (12, 34.56); tup2 = ('abc', 'xyz');
# Following action is not valid for tuples
```

```
# tup1[0] = 100;
# So let's create a new tuple as follows
tup3 = tup1 + tup2;
print tup3;
```

When the above code is executed, it produces the following result −

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the del statement. For example −

Live Demo

```
#!/usr/bin/python

 tup = ('physics', 'chemistry', 1997, 2000);
print tup;
del tup;
print "After deleting tup : ";
print tup;
```

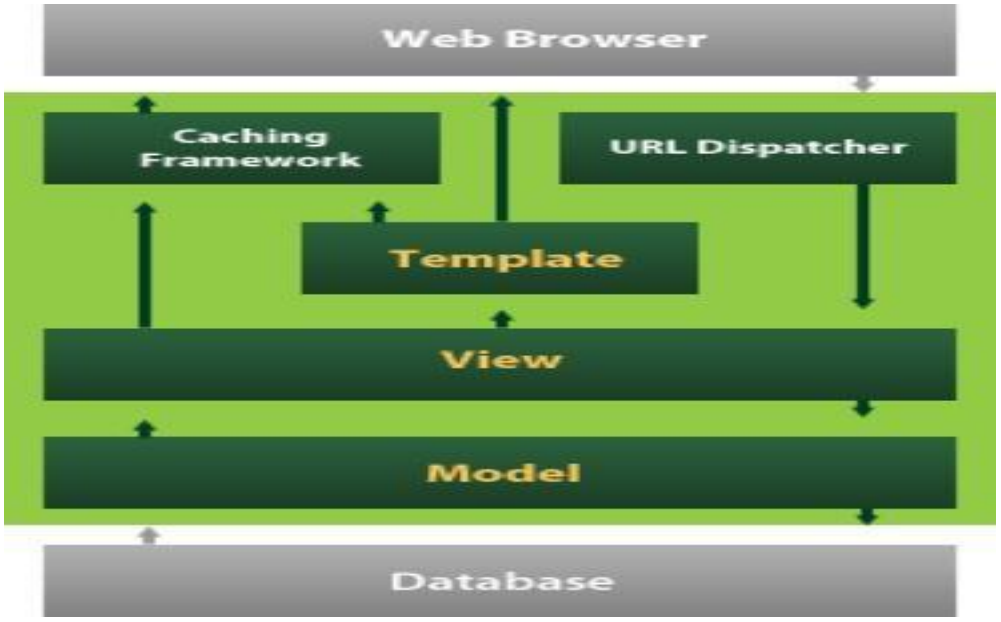
This produces the following result. Note an exception raised, this is because after del tup tuple does not exist any more −

```
('physics', 'chemistry', 1997, 2000)
After deleting tup :
Traceback (most recent call last):
  File "test.py", line 9, in <module>
    print tup;
NameError: name 'tup' is not defined
```

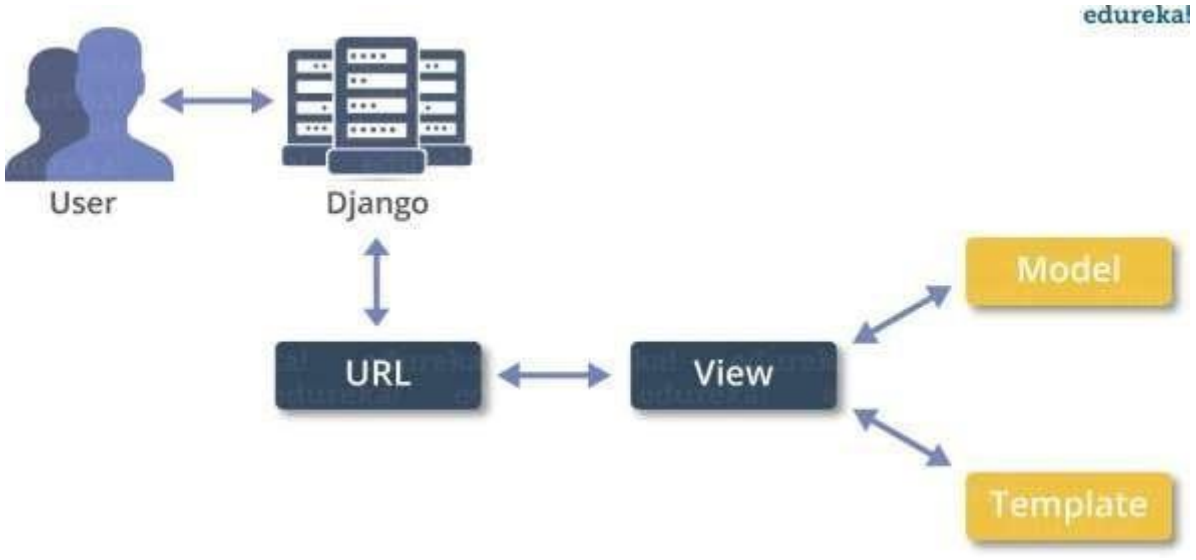
DJANGO

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It's free and open source.

Django's primary goal is to ease the creation of complex, database-driven websites. Django emphasizes reusabilityand "pluggability" of components, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models.



Django also provides an optional administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models



Create a Project

Whether you are on Windows or Linux, just get a terminal or a cmd prompt and navigate to the place you want your project to be created, then use this code −

$ django-admin startproject myproject

This will create a "myproject" folder with the following structure −

myproject/

manage.py

myproject

/

_init____.py

settings.py

urls.py

wsgi.py

The Project Structure

The “myproject” folder is just your project container, it actually contains two elements − manage.py − This file is kind of your project local django-admin for interacting with your project via command line (start the development server, sync db...). To get a full list of command accessible via manage.py you can use the code −

$ python manage.py help

The “myproject” subfolder − This folder is the actual python package of your project. It contains four files −_init_.py − Just for python, treat this folder as package.

settings.py − As the name indicates, your project settings.

urls.py − All links of your project and the function to call. A kind of ToC of your project. wsgi.py − If you need to deploy your project over WSGI.

Setting Up Your Project

Your project is set up in the subfolder myproject/settings.py. Following are some important options you might need to set −

DEBUG = True

This option lets you set if your project is in debug mode or not. Debug mode lets you get more information about your project's error. Never set it to ‘True’ for a live project. However, this has to be set to ‘True’ if you want the Django light server to serve static files. Do it only in the

development mode.

```
DATABASES = {

  'default': {
    'ENGINE':          'django.db.backends.sqlite3',
    'NAME': 'database.sql',
    'USER': '',
    'PASSWORD': '',
    'HOST': '',
    'PORT': '',
  }
}
```

Database is set in the 'Database' dictionary. The example above is for SQLite engine. As stated earlier, Django also supports −

MySQL (django.db.backends.mysql)

PostGreSQL (django.db.backends.postgresql_psycopg2)

Oracle (django.db.backends.oracle) and NoSQL DB

MongoDB (django_mongodb_engine)

Before setting any new engine, make sure you have the correct db driver installed.

You can also set others options like: TIME_ZONE, LANGUAGE_CODE, TEMPLATE… Now that your project is created and configured make sure it's working −

$ python manage.py runserver

You will get something like the following on running the above code −

Validating models...

0 errors found

September 03, 2015 - 11:41:50

Django version 1.6.11, using settings 'myproject.settings' Starting development server at http://127.0.0.1:8000/ Quit the server with CONTROL-C.

A project is a sum of many applications. Every application has an objective and can be reused into another project, like the contact form on a website can be an application, and can be reused for others. See it as a module of your project.

Create an Application

We assume you are in your project folder. In our main “myproject” folder, the same folder then manage.py −

```
$ python manage.py startapp myapp
```

You just created myapp application and like project, Django create a “myapp” folder with the application structure −

```
myapp/
  _init____.py
  admin.py
  models.py
  tests.py
  views.py
```

_init .py − Just to make sure python handles this folder as a package.

admin.py − This file helps you make the app modifiable in the admin interface.

models.py − This is where all the application models are stored.

tests.py − This is where your unit tests are.

views.py − This is where your application views are.

Get the Project to Know About Your Application

At this stage we have our "myapp" application, now we need to register it with our Django project "myproject". To do so, update INSTALLED_APPS tuple in the settings.py file of your project (add your app name) −

```
INSTALLED_APPS = (
  'django.contrib.admin',
  'django.contrib.auth',
  'django.contrib.contenttypes',
  'django.contrib.sessions',
  'django.contrib.messages',
  'django.contrib.staticfiles',
  'myapp',
)
```

Creating forms in Django, is really similar to creating a model. Here again, we just need to inherit from Django class and the class attributes will be the form fields. Let's add a forms.py file in

myapp folder to contain our app forms. We will create a login form.

myapp/forms.py

```
#-*- coding: utf-8 -*-
from django import forms
class LoginForm(forms.Form):
  user = forms.CharField(max_length = 100)
  password = forms.CharField(widget = forms.PasswordInput())
```

As seen above, the field type can take "widget" argument for html rendering; in our case, we want the password to be hidden, not displayed. Many others widget are present in Django: DateInput for dates, CheckboxInput for checkboxes, etc.

Using Form in a View

There are two kinds of HTTP requests, GET and POST. In Django, the request object passed as parameter to your view has an attribute called "method" where the type of the request is set, and all data passed via POST can be accessed via the request.POST dictionary.

Let's create a login view in our myapp/views.py −

```
#-*- coding: utf-8 -*-
from myapp.forms import LoginForm
def login(request):
  username = "not logged in"
    if request.method == "POST":
    #Get the posted form
    MyLoginForm = LoginForm(request.POST) if
    MyLoginForm.is_valid():
      username = MyLoginForm.cleaned_data['username']
  else:
    MyLoginForm = Loginform()
 return render(request, 'loggedin.html', {"username" : username})
```

The view will display the result of the login form posted through the loggedin.html. To test it, we will first need the login form template. Let's call it login.html.

```
<html>
  <body>
    <form name = "form" action = "{% url "myapp.views.login" %}"
```

```
method = "POST" >{% csrf_token %}
 <div style = "max-width:470px;">
   <center>
     <input type = "text" style = "margin-left:20%;"
       placeholder = "Identifiant" name = "username" />
   </center>
 </div>
 <br>
 <div style = "max-width:470px;">
   <center>
     <input type = "password" style = "margin-left:20%;"
       placeholder = "password" name = "password" />
   </center>
 </div>
   <br>
 <div style = "max-width:470px;">
   <center>
     <button style = "border:0px; background-color:#4285F4; margin-top:8%;
       height:35px; width:80%;margin-left:19%;" type = "submit"
       value = "Login" >
       <strong>Login</strong>
     </button>
     </center>
 </div>
   </form>
</body>
</html>
```

The template will display a login form and post the result to our login view above. You have probably noticed the tag in the template, which is just to prevent Cross-site Request Forgery (CSRF) attack on your site.

{% csrf_token %}

Once we have the login template, we need the loggedin.html template that will be rendered after form treatment.

```
<html>
   <body>
    You are : <strong>{{username}}</strong>
  </body>
</html>
```

Now, we just need our pair of URLs to get started: myapp/urls.py

```
from django.conf.urls import patterns, url
from django.views.generic import TemplateView
urlpatterns = patterns('myapp.views',
  url(r'^connection/',TemplateView.as_view(template_name = 'login.html')),
  url(r'^login/', 'login', name = 'login'))
```

When accessing "/myapp/connection", we will get the following login.html template rendered

− Setting Up Sessions

In Django, enabling session is done in your project settings.py, by adding some lines to the MIDDLEWARE_CLASSES and the INSTALLED_APPS options. This should be done while creating the project, but it's always good to know, so MIDDLEWARE_CLASSES should have − 'django.contrib.sessions.middleware.SessionMiddleware'

And INSTALLED_APPS should have − 'django.contrib.sessions'

By default, Django saves session information in database (django_session table or collection), but you can configure the engine to store information using other ways like: in file or in cache.

When session is enabled, every request (first argument of any view in Django) has a session (dict) attribute.

Let's create a simple sample to see how to create and save sessions. We have built a simple login system before (see Django form processing chapter and Django Cookies Handling chapter). Let us save the username in a cookie so, if not signed out, when accessing our login page you won't see the login form. Basically, let's make our login system we used in Django Cookies handling more secure, by saving cookies server side.

For this, first lets change our login view to save our username cookie server side

−

```
def login(request):
  username = 'not logged in'
```

```
  if request.method == 'POST':
    MyLoginForm = LoginForm(request.POST) if
  MyLoginForm.is_valid():
      username = MyLoginForm.cleaned_data['username']
     request.session['username'] = username
    else:
 MyLoginForm = LoginForm()
 return render(request, 'loggedin.html', {"username" : username}
```

Then let us create formView view for the login form, where we won't display the form if cookie is set −

```
def formView(request):
  if request.session.has_key('username'):
    username = request.session['username']
    return render(request, 'loggedin.html', {"username" : username})
  else:
    return render(request, 'login.html', {})
```

Now let us change the url.py file to change the url so it pairs with our new view −

```
from django.conf.urls import patterns, url
from django.views.generic import TemplateView
urlpatterns = patterns('myapp.views',
  url(r'^connection/','formView', name = 'loginform'),
  url(r'^login/', 'login', name = 'login'))
```

When accessing /myapp/connection, you will get to see the following page

7. IMPLEMENTATION

7.1 MODULES

- Manager
- user
- Admin
- Machine-learning

7.2 Modules Description

Manager

Manager information and task descriptions for the entire experiment.Manager can upload the file into the database.we can upload the file with file type and name of the file and also particular url to the file to get the information about the file.

User

User information and task descriptions for the entire experiment. User after login into the session he will get two options.he can search the whateverparticular url or information.we can search the particular file and also we can get the weight and rank of the file by using the tf idf concept.

Admin

Admin will give authority to managers and users. In order to facilitate activate the managers and activate the users. the admin can see the details of all users and managers. Admin can get the accuracy results of svm and xgboost algorithms.

Machine learning

Machine learning refers to the computer's acquisition of a kind of ability to make predictive judgments and make the best decisions by analyzing and learning a large number of existing data. The representation algorithms include deep learning, artificial neural networks, decision trees, enhancement algorithms and so on. The key way for computers to acquire artificial intelligence is machine learning. Nowadays, machine learning plays an important role in various fields of artificial intelligence. Whether in aspects of internet search, biometric identification, auto driving, Mars robot, or in American presidential election, military decision assistants and so on, basically,

as long as there is a need for data analysis, machine learning can be used to play a role.

7.3 Sample Code

url.py

```
 path('userlogin/',user.userlogin,name='userlogin'),
path('userregister/',user.userregister,name='userregister'),
path('userlogincheck/',user.userlogincheck,name='userlogincheck'),
path('pagerank',user.pagerank,name='pagerank'),
path('search/',user.search, name="search"),
path('search1/',user.search1, name="search1"),
path('usersearchresult/',user.usersearchresult, name="usersearchresult"),
path('usersearchresult1/',user.usersearchresult1, name="usersearchresult1"),
path('weight/', user.weight, name="weight"),
path('logout/',user.logout,name='logout'),
path('managerlogin/',manager.managerlogin,name='managerlogin'),
path('managerregister/',manager.managerregister,name='managerregister'),
path('managerlogincheck/',manager.managerlogincheck,name='managerlogincheck'),
path('fileupload/', manager.fileupload, name='fileupload'),\
path('admin1/',search.adminlogin,name='admin1'),
path('adminloginentered/',search.adminloginentered,name='adminloginentered'),
path('userdetails/',search.userdetails,name='userdetails'),
path('Managerdetails/',search.managerdetails,name='Managerdetails'),
path('activateuser/',search.activateuser,name='activateuser'),
path('activatemanager/',search.activatemanager,name='activatemanager')
```

views.py

```
def managerlogin(request):
    return render(request,'manager/managerlogin.html')
def managerregister(request):
    if request.method=='POST':
        form1=managerForm(request.POST)
        if form1.is_valid():
```

```
            form1.save()
            print("succesfully saved the data")
return render(request, 'manager/managerlogin.html')
            #return HttpResponse("registreration succesfully completed")
        else:
            print("form not valied")
            return HttpResponse("form not valied")
    else:
        form=managerForm()
        return
 render(request,"manager/managerregister.html",{"form":form}) def
 managerlogincheck(request):
    if request.method == 'POST':
        sname = request.POST.get('email')
        print(sname)
        spasswd = request.POST.get('upasswd')
        print(spasswd)
        try:
            check = managerModel.objects.get(email=sname,passwd=spasswd)
            # print('usid',usid,'pswd',pswd)
            print(check)
            # request.session['name'] =
            check.name #
            print("name",check.name)
          status =
            check.status
            print('status',statu
            s)
            if status == "Activated":
                request.session['email'] = check.email
                return render(request,
            'manager/managerpage.html') else:
```

```
            messages.success(request, 'manager is not
            activated') return render(request,
            'manager/managerlogin.html')
        except Exception as e:
            print('Exception is ',str(e))
            pass
        messages.success(request,'Invalid name and password')
        return render(request,'manager/managerlogin.html')
```

models.py

```
from django.db import models
class userModel(models.Model):
    name = models.CharField(max_length=50)
    email = models.EmailField()
    passwd = models.CharField(max_length=40)
    cwpasswd = models.CharField(max_length=40)
    mobileno = models.CharField(max_length=50, default="", editable=True)
    status = models.CharField(max_length=40, default="", editable=True)
    def _str_(self):
        return self.email
 class Meta:
        db_table='userregister'
class weightmodel(models.Model):
    filename = models.CharField(max_length=100)
    file = models.FileField(upload_to='files/pdfs/')
    weight=models.CharField(max_length=100)
    rank=models.CharField(max_length=100,default="",  editable=False)
    label=models.CharField(max_length=100,default="", editable=False)
def___str_(self):
        return self.filename
    class Meta:
        db_table='weight’
```

forms.py

```
from django import forms
from user.models import
*
from django.core import validators
class userForm(forms.ModelForm):
    name = forms.CharField(widget=forms.TextInput(), required=True, max_length=100,)
    passwd = forms.CharField(widget=forms.PasswordInput(), required=True, max_length=100)
    cwpasswd = forms.CharField(widget=forms.PasswordInput(), required=True,
max_length=100)
    email = forms.CharField(widget=forms.TextInput(),required=True)
    mobileno= forms.CharField(widget=forms.TextInput(), required=True,
max_length=10,validators=[validators.MaxLengthValidator(10),validators.MinLengthValidator
(10)])
    status = forms.CharField(widget=forms.HiddenInput(), initial='waiting', max_length=100)
def__str_(self):
        return self.email
  class Meta:
        model=userModels
fields=['name','passwd','cwpasswd','email','mobileno','status']
```

userdetails.html

```
{% extends 'adminbase.html' %}
{% load static %}
{% block contents %}
<div class="modal fade" id="login" role="dialog">
    <div class="modal-dialog modal-sm">
      <!-- Modal content no 1-->
      <div class="modal-content">
        <div class="modal-header">
          <button type="button" class="close" data-dismiss="modal">&times;</button>
          <h4 class="modal-title text-center form-title">Login</h4>
```

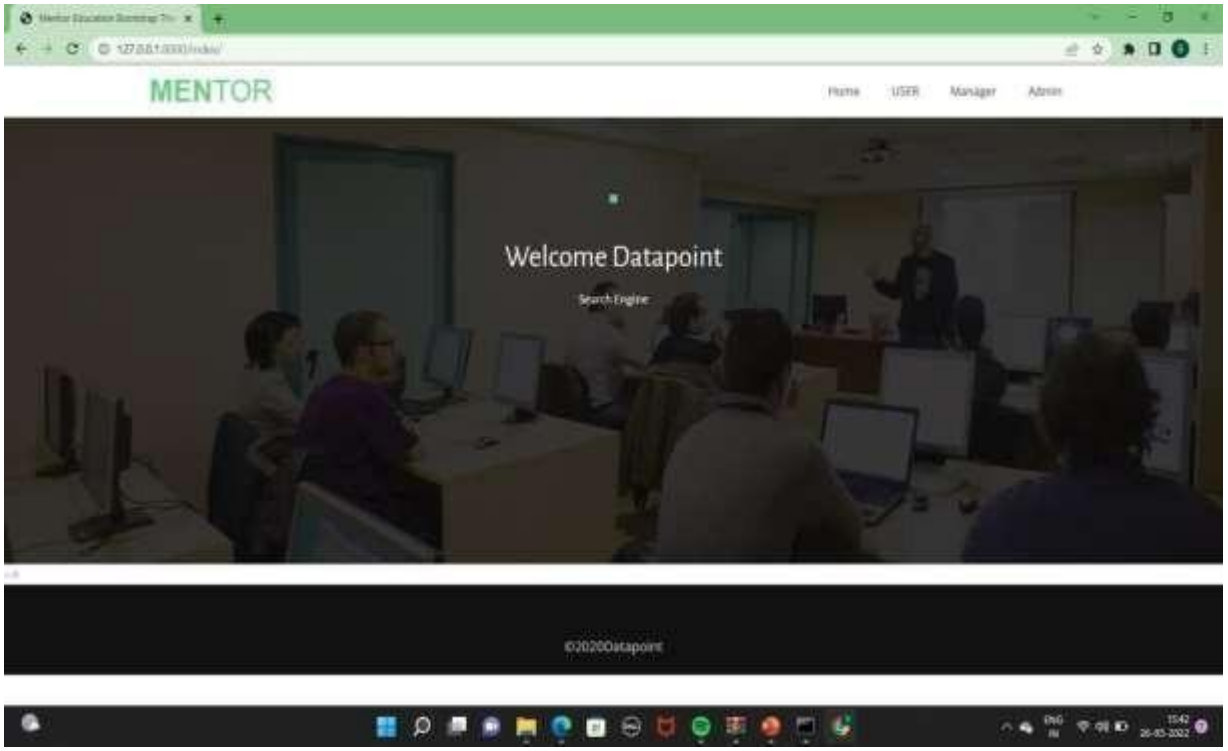
```
</div>
<div class="modal-body padtrbl">
<div class="login-box-body">
   <p class="login-box-msg">Sign in to start your session</p>
   <div class="form-group">
    <form name="" id="loginForm">
     <div class="form-group has-feedback">
      <!.....username.....>
      <input class="form-control" placeholder="Username" id="loginid" type="text"
autocomplete="off" />
      <span style="display:none;font-weight:bold; position:absolute;color: red;position:
absolute;padding:4px;font-size: 11px;background-color:rgba(128, 128, 128, 0.26);z-index: 17;
right: 27px; top: 5px;" id="span_loginid"></span>
      <!---Alredy exists  ! -->
      <span class="glyphicon glyphicon-user form-control-feedback"></span>
     </div>
     <div class="form-group has-feedback">
      <!.....password.....>
      <input class="form-control" placeholder="Password" id="loginpsw"
type="password" autocomplete="off" />
      <span style="display:none;font-weight:bold; position:absolute;color: grey;position:
absolute;padding:4px;font-size: 11px;background-color:rgba(128, 128, 128, 0.26);z-index: 17;
right: 27px; top: 5px;" id="span_loginpsw"></span>
      <!---Alredy exists  ! -->
      <span class="glyphicon glyphicon-lock form-control-feedback"></span>
     </div>
     <div class="row">
      <div class="col-xs-12">
       <div class="checkbox icheck">
        <label>
            <input type="checkbox" id="loginrem" > Remember Me
          </label>
       </div>
```

```
                </div>
                <div class="col-xs-12">
                  <button type="button" class="btn btn-green btn-block btn-flat"
onclick="userlogin()">Sign In</button>
                </div>
              </div>
            </form>
          </div/>
</div>
        </div>
      </div>
    </div>
  </div>
  <!--/ Modal box-->
  <!--Banner-->
  <div class="banner">
    <div class="bg-color">
      <div class="container">
        <div class="row">
          <div class="banner-text text-center">
            <div class="text-border">
              <!--<h2  class="text-dec"></h2>-->
            </div>
            <div class="intro-para text-center quote">
            <p>
           Welcome admin page...
        </p>
              <center><h3>
        <table border="2px solid red" align="left">
                              <tr><th style="color:green">Id</th>
                                 <th style="color:green">name</th>
                                <th style="color:green">status</th>
                                <th style="color:green">activate</th>
```

```
                    </tr>
                   {% for x in qs %}
                    <tr>
                      <td style="color:red">{{x.id}}</td>
                      <td style="color:red">{{x.name}}</td>
                       <td style="color:red">{{x.email}}</td>
                      <td style="color:red">{{x.mobileno}}</td>
                      <td style="color:red">{{x.status}}</td>
                       {% if x.status == 'waiting' %}
                      <td style="color:orange"> <a href="/activateuser/?pid={{ x.id }}"
>Activate</a></td>
                      {% else %}
                            <td style="color:orange"> Activated</td>
                            {% endif %}
                      </tr>
                      {% endfor %}
                </table
                >
              </h3>
            </center>
          </div></a
```

8. SCREENSHOTS

Home Page



User Registration

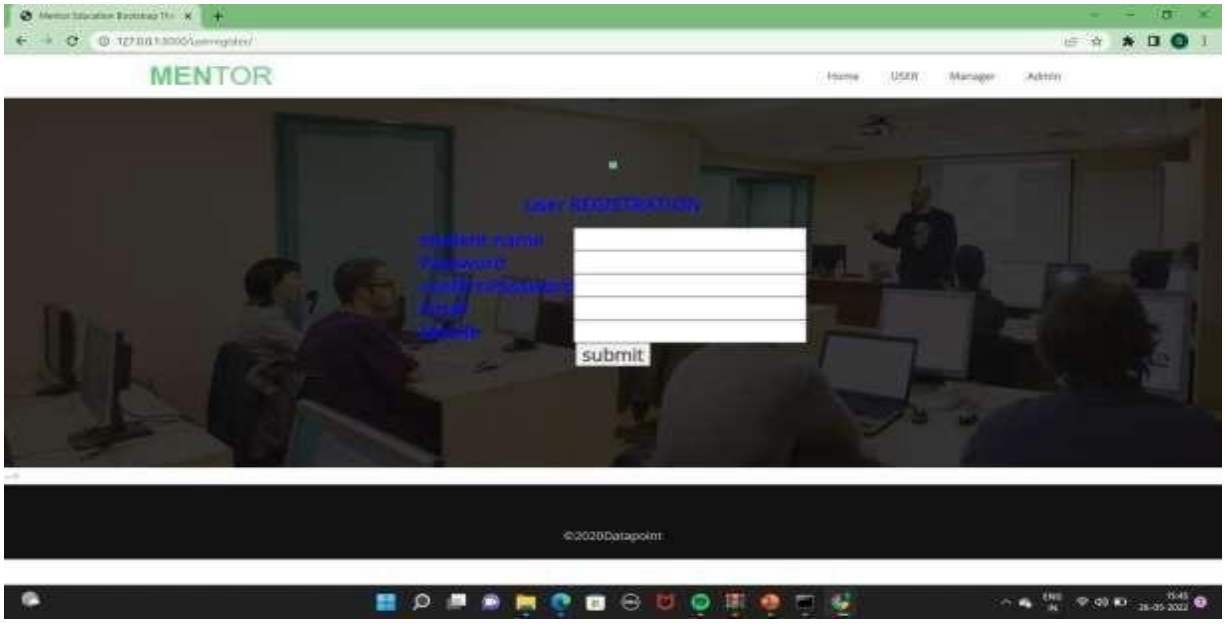
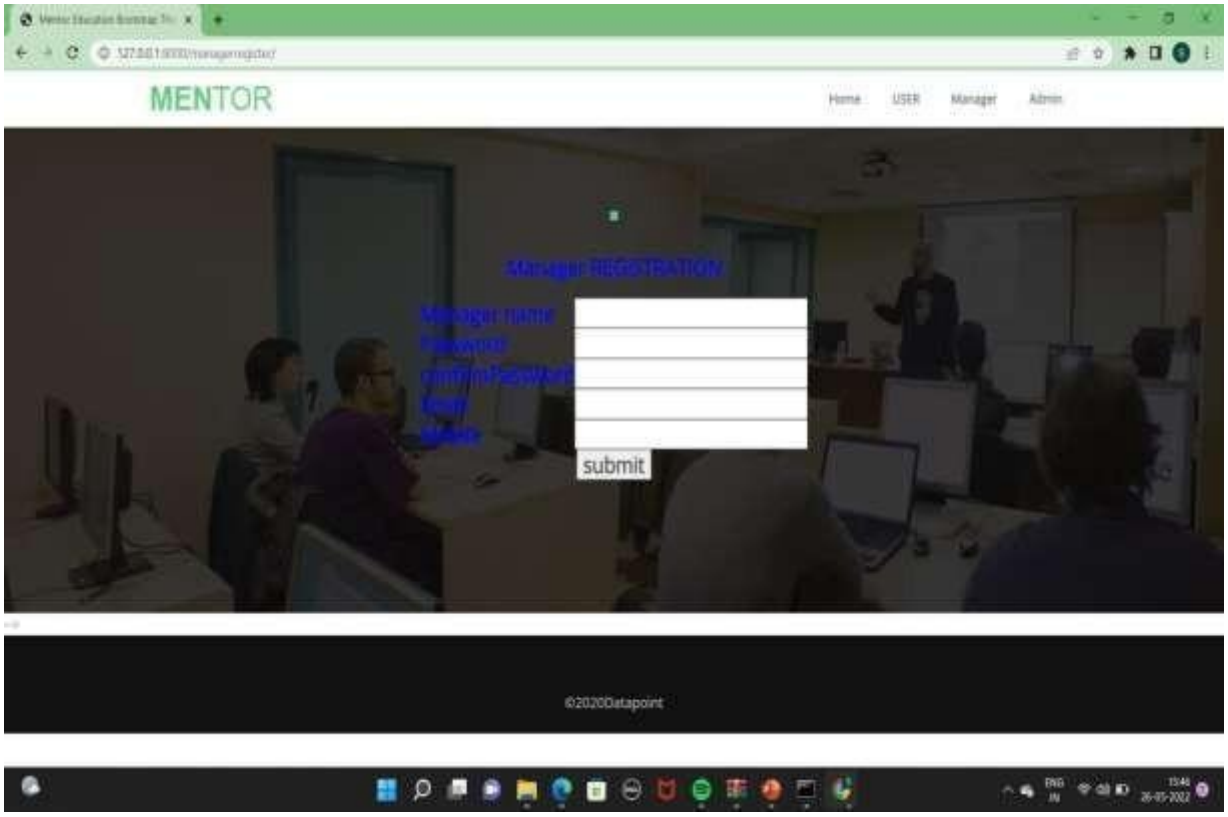


Fig No. 8.1

Manager Registration



Manager Details

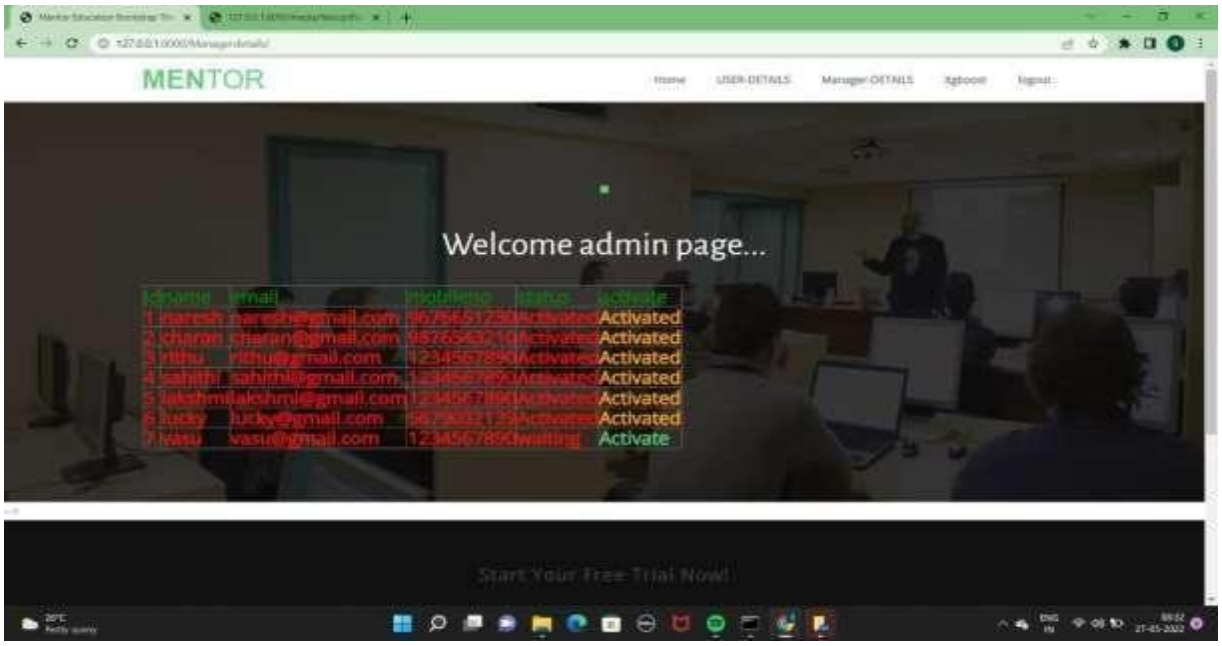


Fig No. 8.2

Manager Login

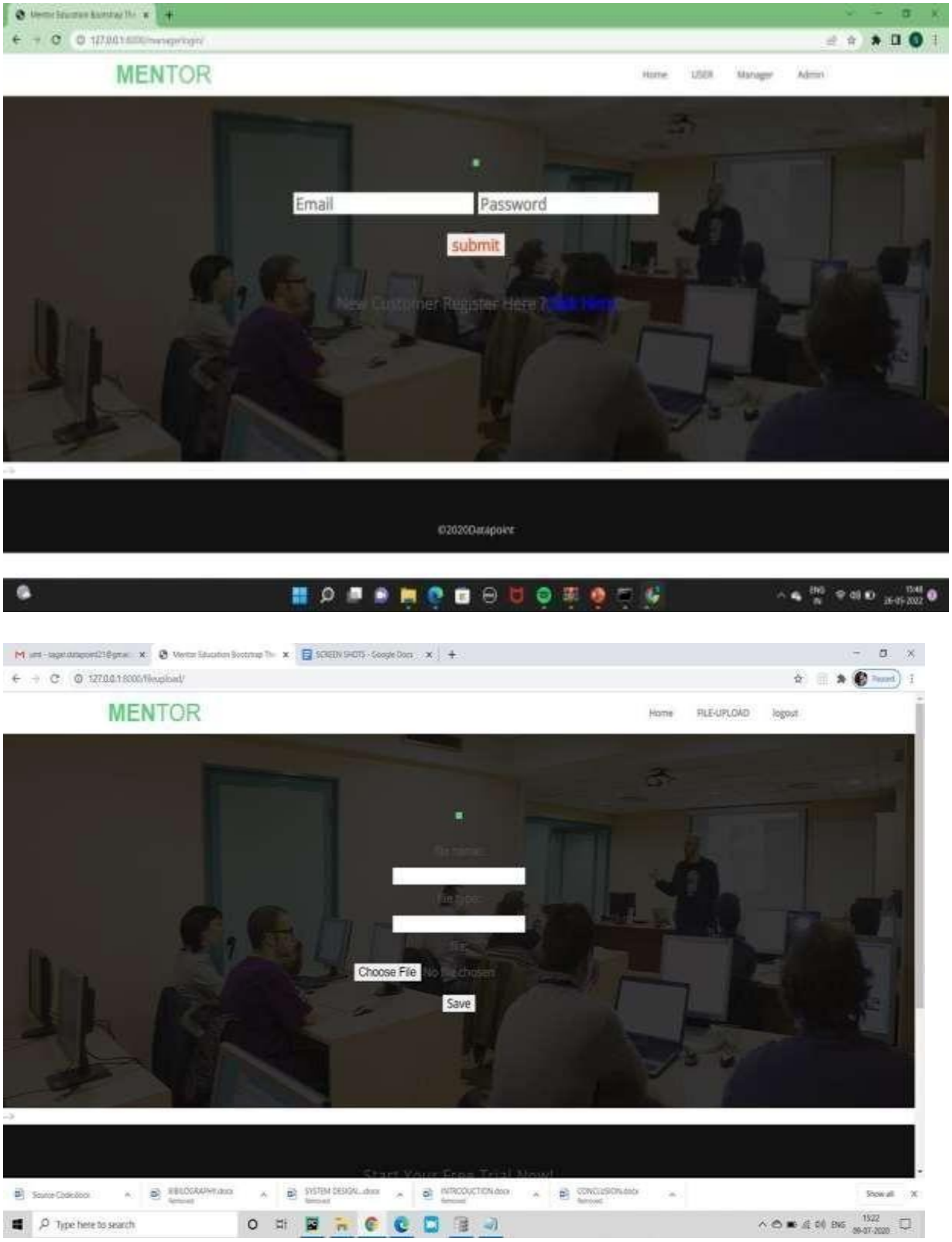
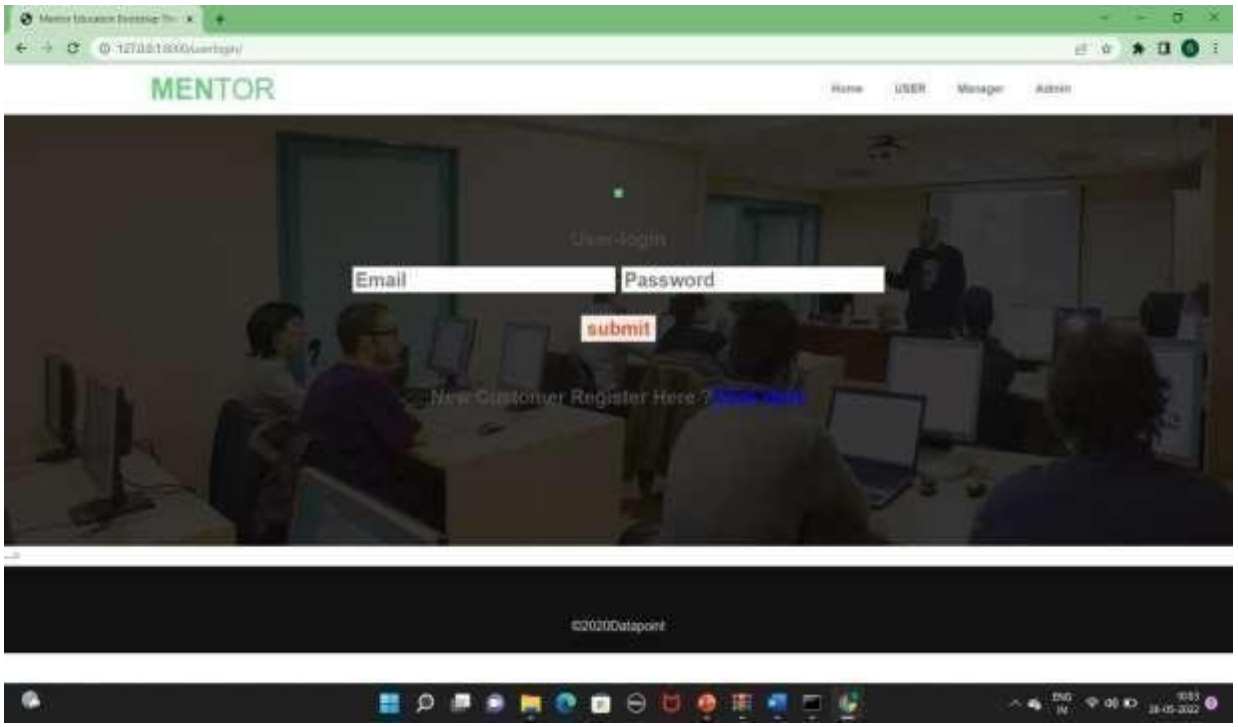


Fig No. 8.3

User Login



User Home

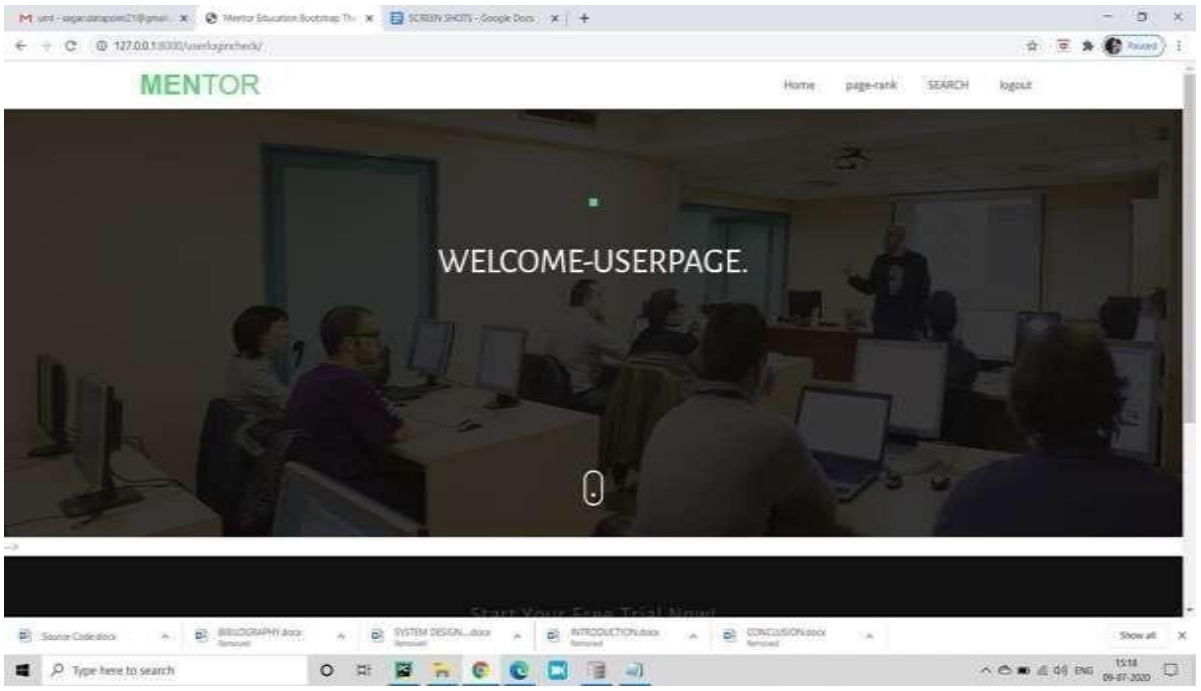
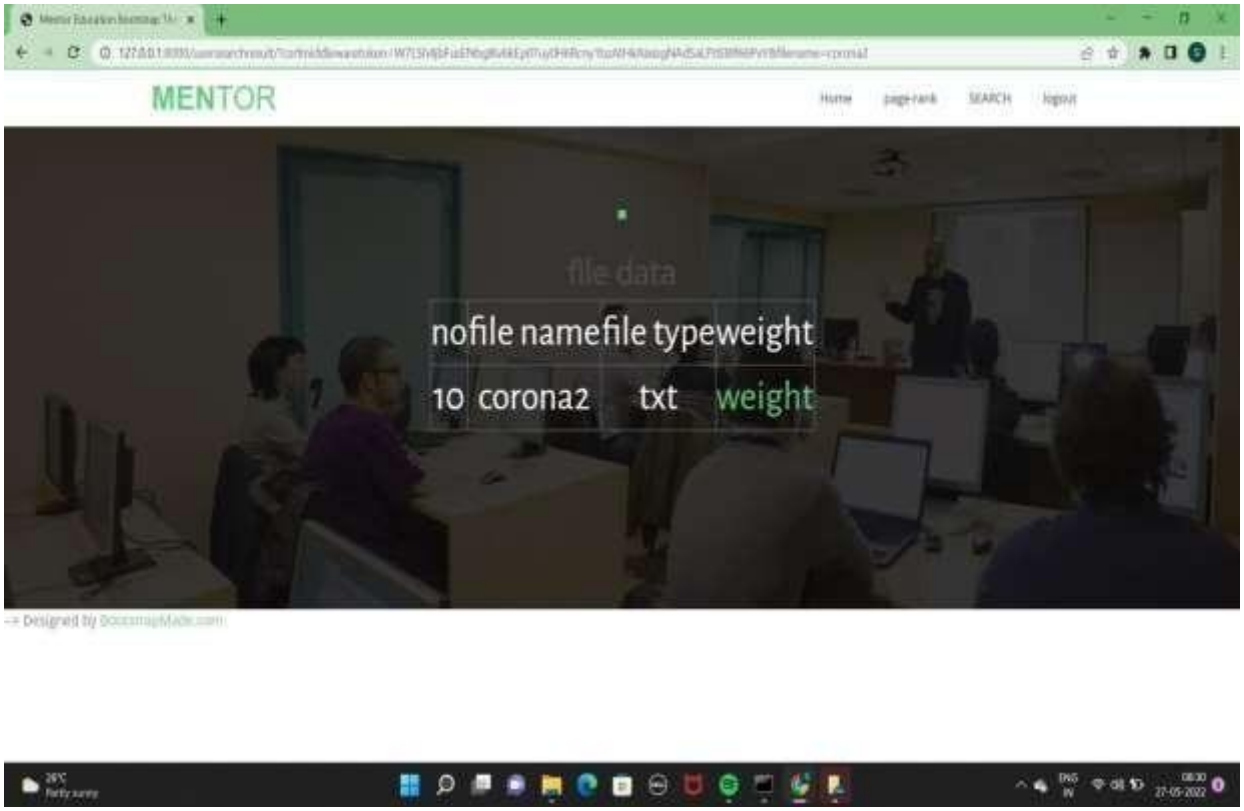


Fig No. 8.4


MENTOR
Home
page-rank
SEARCH
logout
file data
nofile namefile typeweight
10 corona2 txt weight

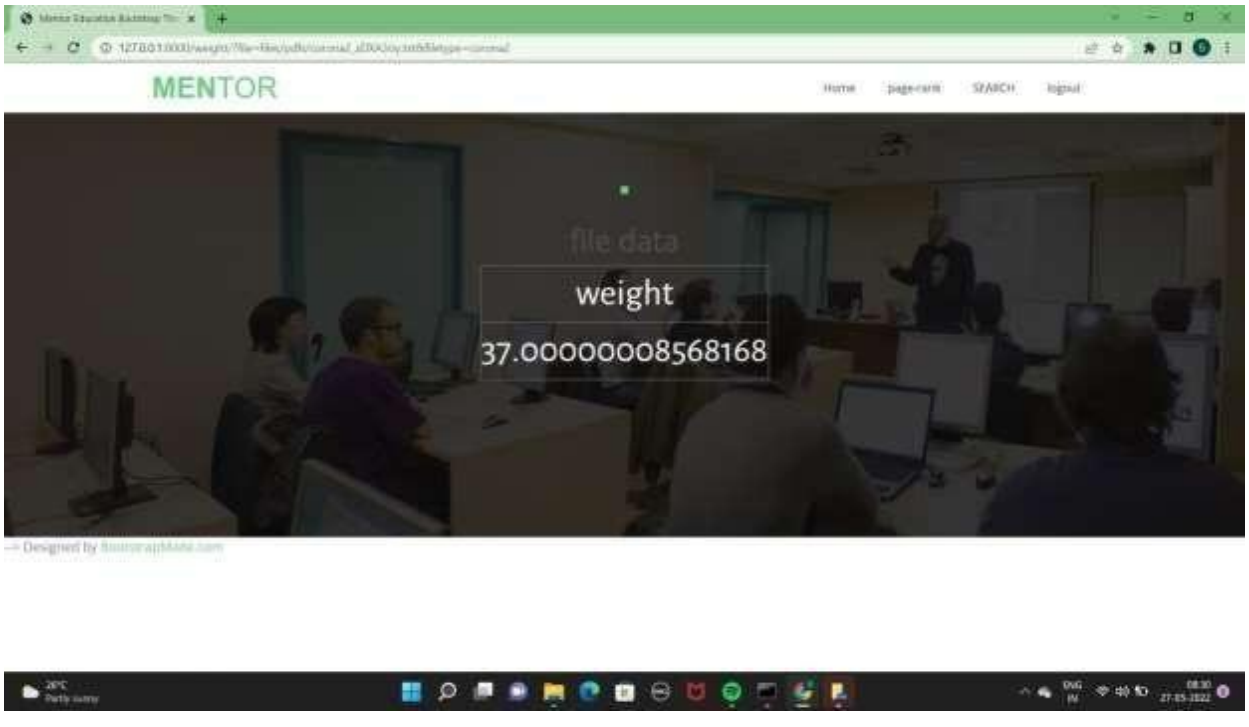
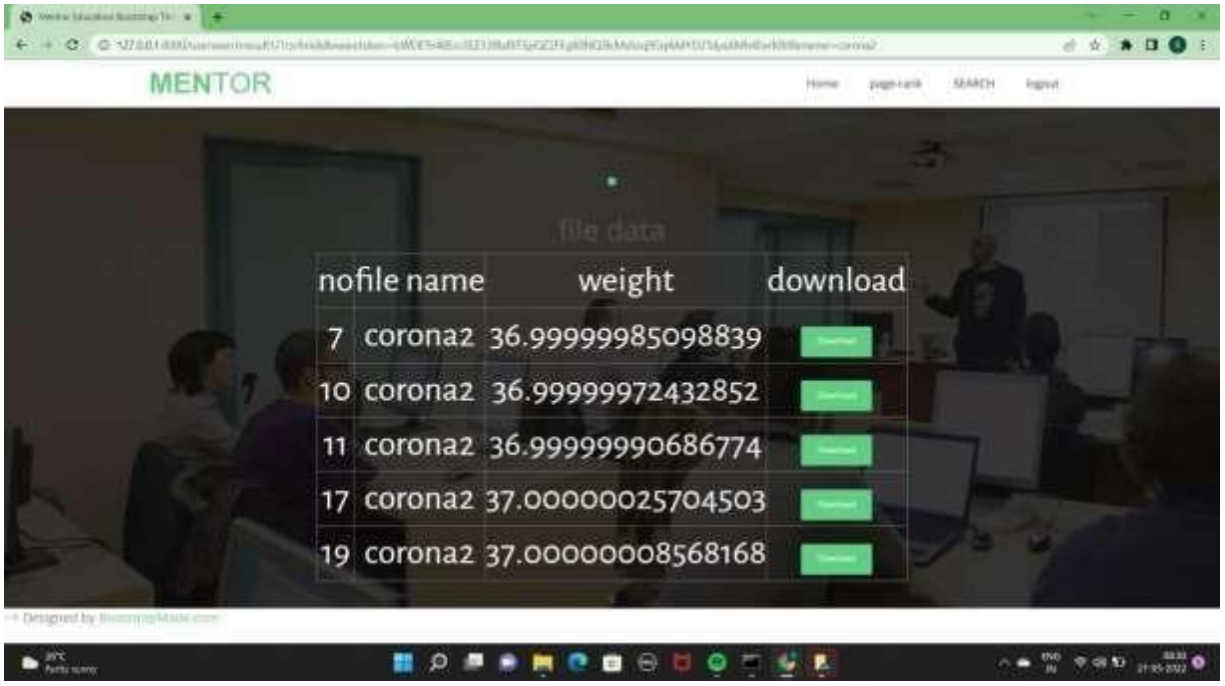

MENTOR
Home
page-rank
SEARCH
logout
file data
weight
37.000000008568168

Fig No. 8.5


MENTOR
Home
page rank
SEARCH
logout
file data
nofile name
weight
download
7 corona2 36.99999985098839
10 corona2 36.99999972432852
11 corona2 36.99999990686774
17 corona2 37.00000025704503
19 corona2 37.00000008568168

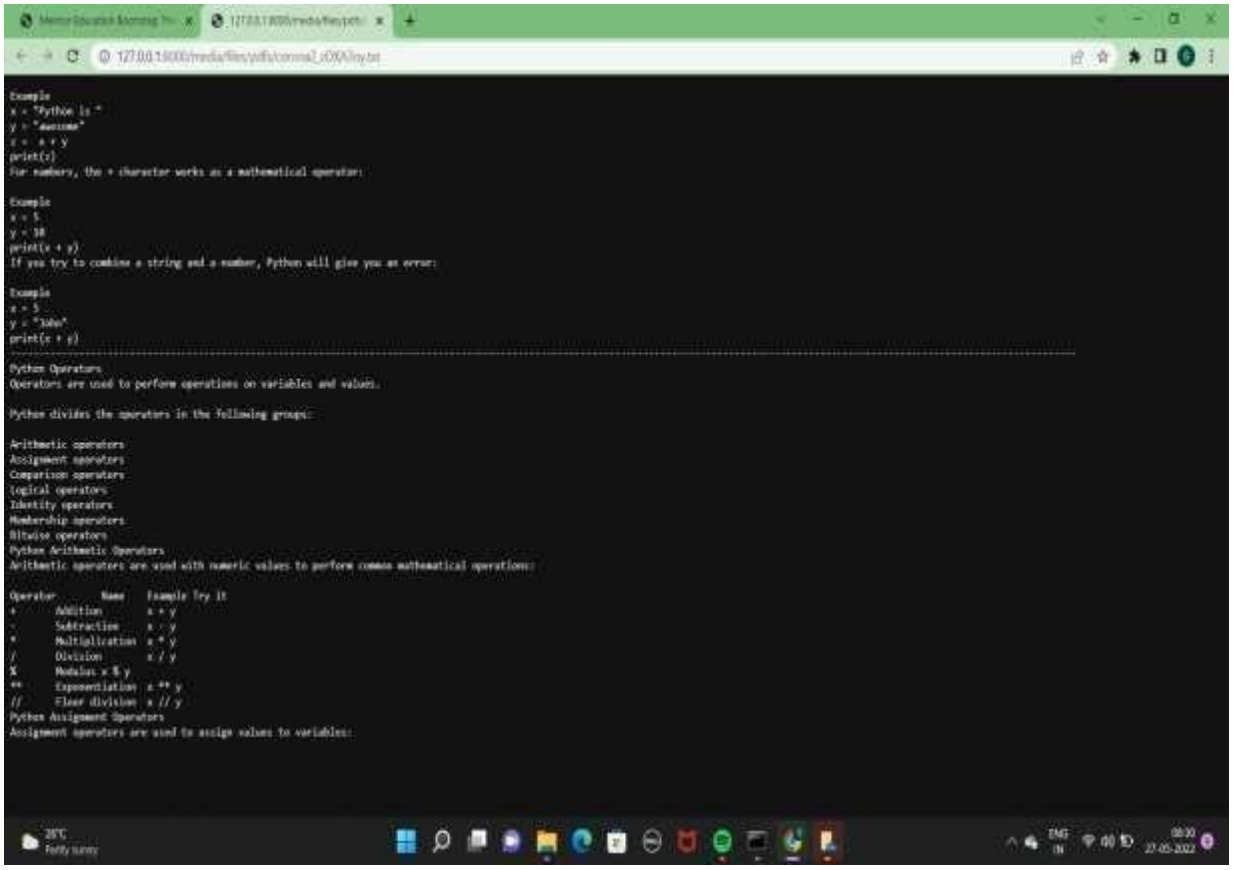
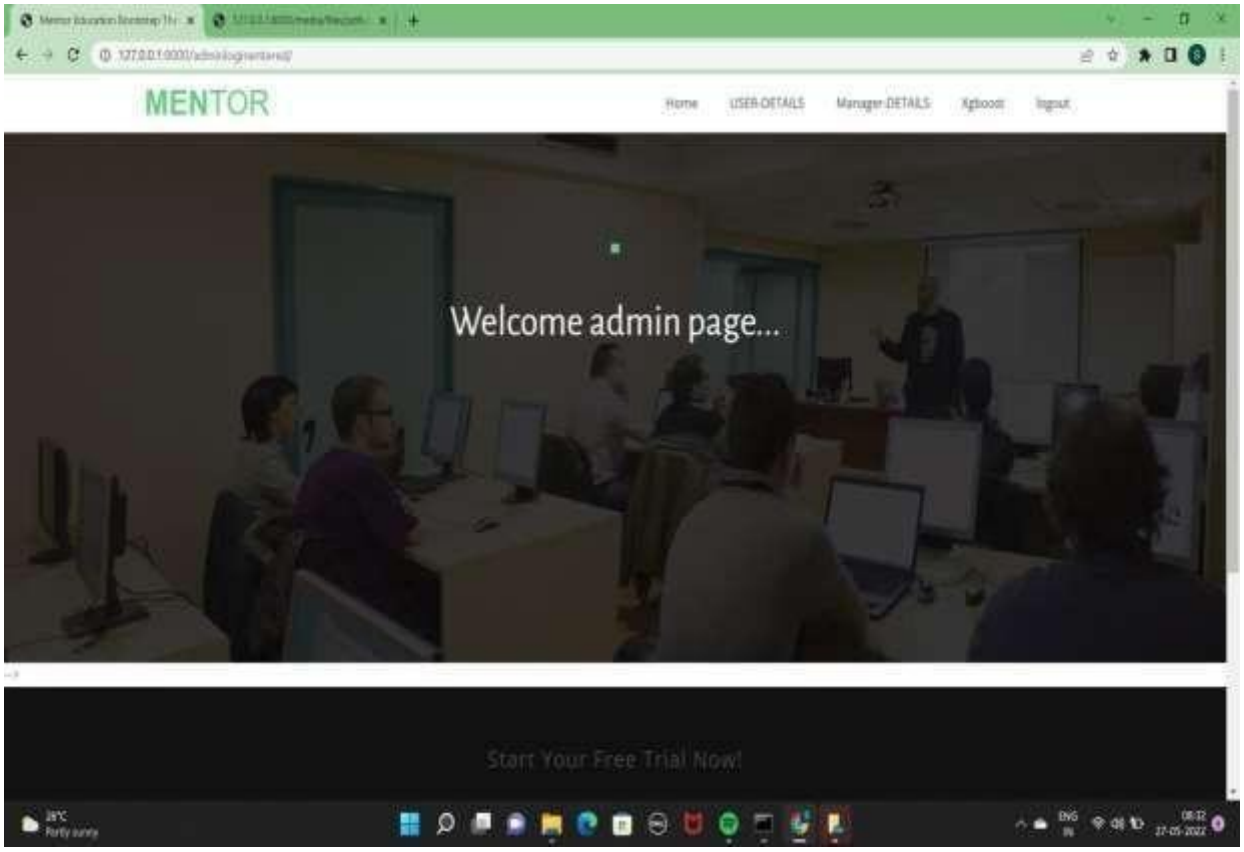

Example
x = "Python is "
y = "awesome"
z = x + y
print(z)
For numbers, the + character works as a mathematical operator:
Example
x = 5
y = 10
print(x + y)
If you try to combine a string and a number, Python will give you an error:
Example
x = 5
y = "John"
print(x + y)
Python Operators
Operators are used to perform operations on variables and values.
Python divides the operators in the following groups:
Arithmetic operators
Assignment operators
Comparison operators
Logical operators
Identity operators
Membership operators
Bitwise operators
Python Arithmetic Operators
Arithmetic operators are used with numeric values to perform common mathematical operations:
Operator Name Example Try it
+ Addition x + y
- Subtraction x - y
* Multiplication x * y
/ Division x / y
% Modulus x % y
** Exponentiation x ** y
// Floor division x // y
Python Assignment Operators
Assignment operators are used to assign values to variables:

Fig No. 8.6


MENTOR
Home
USER-DETAILS
Manager-DETAILS
Xgboost
logout
Welcome admin page...
Start Your Free Trial Now!

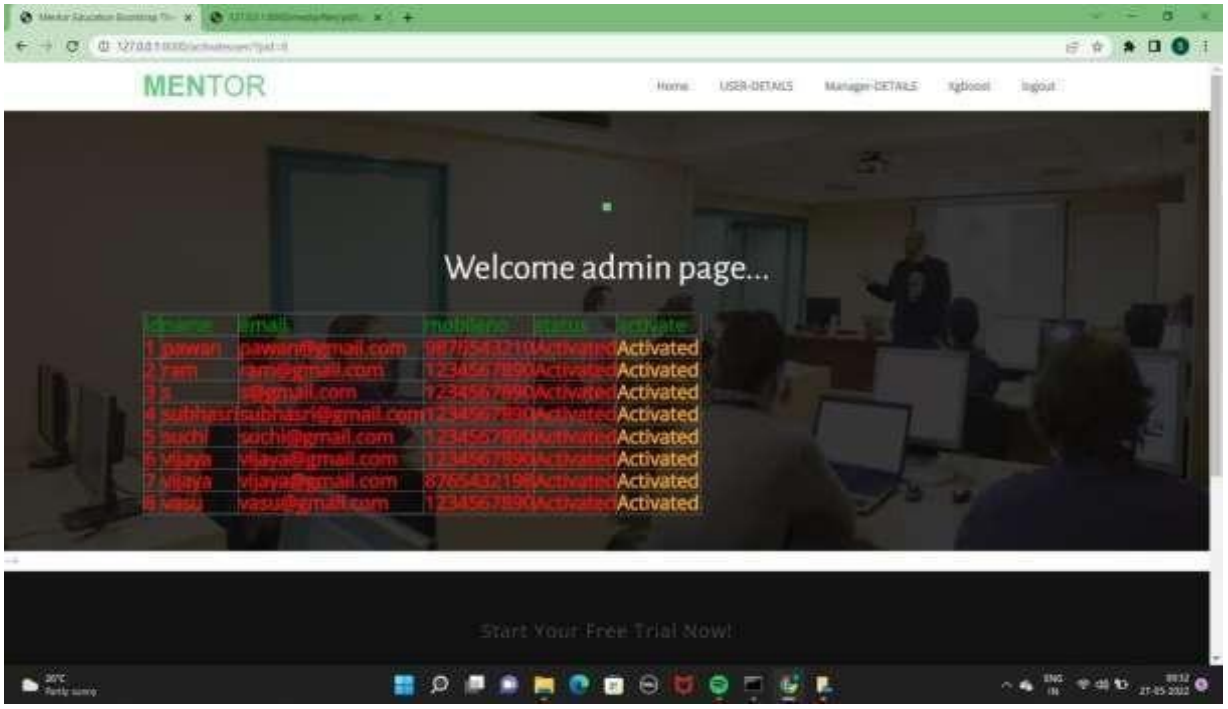

MENTOR
Home
USER-DETAILS
Manager-DETAILS
Xgboost
logout
Welcome admin page...
Activated
Start Your Free Trial Now!

Fig No. 8.7

9. TESTING & VALIDATION

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

9.1 Types of Test

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

9.2 Levels of Tests

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the

software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level –interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements. **Test Results:** All the test cases mentioned above passed successfully. No defects encountered.

9.3 Test Results

S.no	Test Case	Excepted Result	Result	Remarks(IF Fails)
1.	User Register	If User registratio n successfully.	Pass	If already user email exists then it fails.
2.	User Login	If the Username and password is correct then it will be a valid page.	Pass	Unknown Register Users will not be logged in.
3.	Manager login	If the Manager name and password is correct then it will be a valid pag.	Pass	.Unknown Register Manager will not log in.
4.	Admin can activate the register magers	Admin can activate the register manager id.	Pass	If the manager did not find it then it won't login
5.	Admin login	Admin can login with his login credential. If success he get is home page	Pass	Invalid login details will not allowed here

6.	Admin can activate the register users	Admin can activate the register user id .	Pass	.If the user did not find it then it won't login.
7.	admin can get the svm results	by clicking svm it will display svm prediction	Pass	prediction of svm won't get..
8.	admin can get the xgboost results	by clicking xgboost it will display xgboost prediction.	Pass	prediction of xgboost won't get..
9.	user login page	user can search the weight of particular documen t	Pass	we won't get the weight of document.

10. CONCLUSION AND FUTURE SCOPE

Search engines are very useful for finding out more relevant URLs for given keywords. Due to this, user time is reduced for searching the relevant web page. For this, Accuracy is a very important factor. From the above observation, it can be concluded that XGBoost is better in terms of accuracy than SVM and ANN. Thus, Search engines built using XGBoost and PageRank algorithms will give better accuracy.

A large-scale web search engine is a complex system and much remains to be done. Our immediate goals are to improve search efficiency and to scale to approximately 100 million web pages.

REFERENCES

[1] Manika Dutta, K. L. Bansal, "A Review Paper on Various Search Engines (Google, Yahoo, Altavista, Ask and Bing)", International Journal on Recent and Innovation Trends in Computing and Communication, 2016.

[2] Gunjan H. Agre, Nikita V.Mahajan, "Keyword Focused Web Crawler", International Conference on Electronic and Communication Systems, IEEE, 2015.

[3] Tuhena Sen, Dev Kumar Chaudhary, "Contrastive Study of Simple PageRank, HITS and Weighted PageRank Algorithms: Review", International Conference on Cloud Computing, Data Science & Engineering, IEEE, 2017.

[4] Michael Chau, Hsinchun Chen, "A machine learning approach to web page filtering using content and structure analysis", Decision Support Systems 44 (2008) 482–494,scienceDirect,2008.

[5] Taruna Kumari, Ashlesha Gupta, Ashutosh Dixit, "Comparative Study of Page Rank and Weighted Page Rank Algorithm", International Journal of Innovative Research in Computer and Communication Engineering, February 2014.

[6] K. R. Srinath, "Page Ranking Algorithms – A Comparison", International Research Journal of Engineering and Technology (IRJET), Dec2017.

[7] S. Prabha, K. Duraiswamy, J. Indhumathi, "Comparative Analysis of Different Page Ranking Algorithms", International Journal of Computer and Information Engineering, 2014.

[8] Dilip Kumar Sharma, A. K. Sharma, "A Comparative Analysis of Web Page Ranking Algorithms", International Journal on Computer Science and Engineering, 2010.

[9] Vijay Chauhan, Arunima Jaiswal, Junaid Khalid Khan, "Web Page Ranking Using Machine Learning Approach", International Conference on Advanced Computing Communication Technologies, 2015.

[10] Amanjot Kaur Sandhu, Tiewei s. Liu., "Wikipedia Search Engine: Interactive Information Retrieval Interface Design", International Conference on Industrial and Information Systems, 2014.

[11] Neha Sharma, Rashi Agarwal, Narendra Kohli, "Review of features and machine learning techniques for web searching", International Conference on Advanced Computing Communication Technologies,2016.

[12] Sweah Liang Yong, Markus Hagenbuchner, Ah Chung Tsoi, "Ranking Web Pages using

Machine Learning Approaches", International Conference on Web Intelligence and Intelligent Agent Technology, 2008.

[13] B. Jaganathan, Kalyani Desikan,"WeigIndian Journal of Science and Technology, Dec-2015hted Page Rank Algorithm based on In-Out Weight of Webpages", Indian Journal of Science and Technology, Dec-2015.

Appendix A:Sample Code

url.py

```
 path('userlogin/',user.userlogin,name='userlogin'),
path('userregister/',user.userregister,name='userregister'),
path('userlogincheck/',user.userlogincheck,name='userlogincheck'),
path('pagerank',user.pagerank,name='pagerank'),
path('search/',user.search, name="search"),
path('search1/',user.search1, name="search1"),
path('usersearchresult/',user.usersearchresult, name="usersearchresult"),
path('usersearchresult1/',user.usersearchresult1, name="usersearchresult1"),
path('weight/', user.weight, name="weight"),
path('logout/',user.logout,name='logout'),
path('managerlogin/',manager.managerlogin,name='managerlogin'),
path('managerregister/',manager.managerregister,name='managerregister'),
path('managerlogincheck/',manager.managerlogincheck,name='managerlogincheck'),
path('fileupload/', manager.fileupload, name='fileupload'),\
path('admin1/',search.adminlogin,name='admin1'),
path('adminloginentered/',search.adminloginentered,name='adminloginentered'),
path('userdetails/',search.userdetails,name='userdetails'),
path('Managerdetails/',search.managerdetails,name='Managerdetails'),
path('activateuser/',search.activateuser,name='activateuser'),
path('activatemanager/',search.activatemanager,name='activatemanager')
```

views.py

```
def managerlogin(request):
   return render(request,'manager/managerlogin.html')
def managerregister(request):
   if request.method=='POST':
      form1=managerForm(request.POST)
      if form1.is_valid():
```

```
            form1.save()
            print("succesfully saved the data")

return render(request, 'manager/managerlogin.html')
            #return HttpResponse("registreration succesfully completed")
        else:
            print("form not valied")
            return HttpResponse("form not valied")
    else:
        form=managerForm()
        return
 render(request,"manager/managerregister.html",{"form":form}) def
 managerlogincheck(request):
    if request.method == 'POST':
        sname = request.POST.get('email')
        print(sname)
        spasswd = request.POST.get('upasswd')
        print(spasswd)
        try:
            check = managerModel.objects.get(email=sname,passwd=spasswd)
            # print('usid',usid,'pswd',pswd)
            print(check)
            # request.session['name'] =
            check.name #
            print("name",check.name)
          status =
            check.status
            print('status',statu
            s)
            if status == "Activated":
                request.session['email'] = check.email
                return render(request,
            'manager/managerpage.html') else:
```

```
            messages.success(request, 'manager is not
            activated') return render(request,
            'manager/managerlogin.html')
        except Exception as e:
            print('Exception is ',str(e))
            pass
        messages.success(request,'Invalid name and password')
        return render(request,'manager/managerlogin.html')
```

models.py

```
from django.db import models
class userModel(models.Model):
    name = models.CharField(max_length=50)
    email = models.EmailField()
    passwd = models.CharField(max_length=40)
    cwpasswd = models.CharField(max_length=40)
    mobileno = models.CharField(max_length=50, default="", editable=True)
    status = models.CharField(max_length=40, default="", editable=True)
    def _str_(self):
        return self.email
 class Meta:
        db_table='userregister'
class weightmodel(models.Model):
    filename = models.CharField(max_length=100)
    file = models.FileField(upload_to='files/pdfs/')
    weight=models.CharField(max_length=100)
    rank=models.CharField(max_length=100,default="",  editable=False)
    label=models.CharField(max_length=100,default="", editable=False)
def__str_(self):
        return self.filename
    class Meta:
        db_table='weight’
```

forms.py

```
from django import forms
from user.models import
*
from django.core import validators

class userForm(forms.ModelForm):
    name = forms.CharField(widget=forms.TextInput(), required=True, max_length=100,)
    passwd = forms.CharField(widget=forms.PasswordInput(), required=True, max_length=100)
    cwpasswd = forms.CharField(widget=forms.PasswordInput(), required=True,
max_length=100)
    email = forms.CharField(widget=forms.TextInput(),required=True)
    mobileno= forms.CharField(widget=forms.TextInput(), required=True,
max_length=10,validators=[validators.MaxLengthValidator(10),validators.MinLengthValidator
(10)])
    status = forms.CharField(widget=forms.HiddenInput(), initial='waiting', max_length=100)
def__str_(self):
        return self.email
  class Meta:
        model=userModels
fields=['name','passwd','cwpasswd','email','mobileno','status']
```

userdetails.html

```
{% extends 'adminbase.html' %}
{% load static %}
{% block contents %}
<div class="modal fade" id="login" role="dialog">
    <div class="modal-dialog modal-sm">
      <!-- Modal content no 1-->
      <div class="modal-content">
        <div class="modal-header">
          <button type="button" class="close" data-dismiss="modal">&times;</button>
          <h4 class="modal-title text-center form-title">Login</h4>
```

```
</div>
<div class="modal-body padtrbl">
<div class="login-box-body">
   <p class="login-box-msg">Sign in to start your session</p>
   <div class="form-group">
    <form name="" id="loginForm">
     <div class="form-group has-feedback">
      <!.....username.....>
      <input class="form-control" placeholder="Username" id="loginid" type="text"
autocomplete="off" />
      <span style="display:none;font-weight:bold; position:absolute;color: red;position:
absolute;padding:4px;font-size: 11px;background-color:rgba(128, 128, 128, 0.26);z-index: 17;
right: 27px; top: 5px;" id="span_loginid"></span>
      <!---Alredy exists  ! -->
      <span class="glyphicon glyphicon-user form-control-feedback"></span>
     </div>
     <div class="form-group has-feedback">
      <!.....password.....>
      <input class="form-control" placeholder="Password" id="loginpsw"
type="password" autocomplete="off" />
      <span style="display:none;font-weight:bold; position:absolute;color: grey;position:
absolute;padding:4px;font-size: 11px;background-color:rgba(128, 128, 128, 0.26);z-index: 17;
right: 27px; top: 5px;" id="span_loginpsw"></span>
      <!---Alredy exists  ! -->
      <span class="glyphicon glyphicon-lock form-control-feedback"></span>
     </div>
     <div class="row">
      <div class="col-xs-12">
       <div class="checkbox icheck">
        <label>
            <input type="checkbox" id="loginrem" > Remember Me
          </label>
       </div>
```

```
                </div>
              <div class="col-xs-12">
                  <button type="button" class="btn btn-green btn-block btn-flat"
onclick="userlogin()">Sign In</button>
                </div>
              </div>
            </form>
          </div/>
</div>
        </div>
      </div>
    </div>
  </div>
  <!--/ Modal box-->
  <!--Banner-->
  <div class="banner">
    <div class="bg-color">
      <div class="container">
        <div class="row">
          <div class="banner-text text-center">
            <div class="text-border">
              <!--<h2  class="text-dec"></h2>-->
            </div>
            <div class="intro-para text-center quote">
            <p>
           Welcome admin page...
        </p>
             <center><h3>
        <table border="2px solid red" align="left">
                          <tr><th style="color:green">Id</th>
                             <th style="color:green">name</th>
                           <th style="color:green">status</th>
                           <th style="color:green">activate</th>
```

```
                    </tr>
                   {% for x in qs %}
                    <tr>
                      <td style="color:red">{{x.id}}</td>
                      <td style="color:red">{{x.name}}</td>
                       <td style="color:red">{{x.email}}</td>
                      <td style="color:red">{{x.mobileno}}</td>
                      <td style="color:red">{{x.status}}</td>
                       {% if x.status == 'waiting' %}
                      <td style="color:orange"> <a href="/activateuser/?pid={{ x.id }}"
>Activate</a></td>
                      {% else %}
                            <td style="color:orange"> Activated</td>
                            {% endif %}
                      </tr>
                      {% endfor %}
               </table
               >
            </h3>
          </center>
        </div></a
```